

4 Module 4: Processes

4.1 Benefits of IPC

4.1.1 1. Introduction to Inter-Process Communication (IPC)

4.1.1.1 1.1 Definition of IPC

- **Inter-Process Communication (IPC)** refers to the mechanisms that enable **cooperating processes** to exchange information and synchronize their actions.
- IPC is **not required for independent processes**, which operate in isolation without affecting or being affected by other processes.
- IPC is **essential for cooperating processes**, which interact with one another to achieve a common goal.

4.1.1.2 1.2 Types of Processes in a Computer System Two primary categories of processes exist in an operating system:

1. Independent Processes

- Do **not** affect other processes.
- Are **not** affected by other processes.
- **Never communicate** with other processes.
- Example: A background system monitor running without interacting with user applications.

2. Cooperating Processes

- **Can affect** other processes.
- **Can be affected** by other processes.
- **Require communication** with other processes to function effectively.
- Example: A web browser (parent process) spawning multiple tabs (child processes) that share data.

4.1.1.3 1.3 Role of IPC in Cooperating Processes

- IPC facilitates **information exchange** between cooperating processes.
- Without IPC, cooperating processes **cannot coordinate** their actions or share resources.
- IPC mechanisms **enable synchronization and data sharing**, ensuring processes work together efficiently.

4.1.2 2. IPC Mechanisms (Overview)

The lecture introduces **three primary IPC mechanisms**, which will be discussed in detail in subsequent lectures:

1. Shared Memory

- Processes share a common memory region.
- Fastest form of IPC (avoids kernel intervention for data transfer).
- Requires synchronization (e.g., semaphores) to prevent race conditions.

2. Message Passing

- Processes exchange messages via the kernel or direct communication.
- More secure than shared memory (no direct memory access).
- Examples: Pipes, message queues, sockets.

3. Pipes

- A **unidirectional** communication channel between processes.
- Data written to one end is read from the other.
- Commonly used in command-line applications (e.g., `ls | grep "file"`).

Note: Detailed explanations of these mechanisms will be covered in later lectures.

4.1.3 3. Benefits of Inter-Process Communication (IPC)

IPC provides several **critical advantages** in modern operating systems, particularly for **cooperating processes** and **distributed systems**.

4.1.3.1 3.1 Cooperative Execution of Processes

- **Task Decomposition:**
 - A **single complex task** can be divided into **multiple subtasks**.
 - Each subtask is assigned to a **separate process**.
- **Parallel Execution:**
 - Subtasks can run **concurrently**, improving efficiency.
 - Example:
 - * A video rendering task may split into:
 1. Audio processing (Process A)
 2. Video encoding (Process B)
 3. Metadata generation (Process C)
 - * IPC ensures these processes **coordinate** to produce the final output.

4.1.3.2 3.2 Resource Sharing

- **Shared Access to Files/Databases:**
 - Multiple processes can **simultaneously access** a shared resource (e.g., a database file, log file, or configuration file).
 - Example:
 - * A **database management system (DBMS)** allows multiple client processes to read/write data concurrently.
 - * A **web server** may share a cache among worker processes to reduce redundant computations.
- **Efficient Utilization:**
 - Avoids **data duplication** (e.g., loading the same file into memory multiple times).
 - Reduces **memory overhead** and **disk I/O**.

4.1.3.3 3.3 Computation Speedup (Parallelism & Throughput)

- **Divide-and-Conquer Approach:**
 - A task is split into **independent subtasks** executed by different processes.
 - If subtasks run **in parallel**, the **total execution time decreases**.
- **Example: Multi-Threaded Applications**
 - A **scientific simulation** may divide computations across multiple CPU cores.
 - A **web server** handles multiple client requests simultaneously via separate processes.
- **Throughput Improvement:**
 - More tasks completed **per unit time** due to parallel execution.
 - Example: A **batch processing system** (e.g., payroll calculation) runs faster when divided among processes.

4.1.3.4 3.4 Support for Distributed Applications

- **Definition of Distributed Applications:**
 - Applications that run across **multiple systems (nodes)** in a network.
 - Each node executes **part of the application** (as separate processes).
- **Mandatory Role of IPC:**
 - Processes on **different machines** must communicate (e.g., via **network sockets** or **RPC**).

- Example:
 - * A **distributed database** (e.g., Cassandra, MongoDB) requires IPC for **data replication** and **consistency**.
 - * A **microservices architecture** relies on IPC (e.g., HTTP/REST, gRPC) for inter-service communication.
- **Without IPC:**
 - Distributed systems **cannot function** as processes would operate in isolation.

4.1.3.5 3.5 Information Sharing

- **Data Propagation Between Processes:**
 - A process may **receive input** (e.g., user data, sensor readings) that other processes need.
 - IPC ensures this information is **shared efficiently**.
- **Example Scenarios:**
 1. **User Input Handling:**
 - A **GUI process** collects user input (e.g., a file path) and shares it with a **background processing task**.
 2. **Real-Time Systems:**
 - A **sensor process** reads temperature data and sends it to a **control process** for adjustments.
 3. **Pipeline Processing:**
 - In a **shell pipeline** (`cat file | grep "error" | sort`), each process passes data to the next via IPC (pipes).

4.1.4 4. Summary of Key Points

Concept	Description
Independent Process	Operates alone; no communication with other processes.
Cooperating Process	Interacts with other processes; requires IPC.
IPC Mechanisms	Shared memory, message passing, pipes (detailed in later lectures).
Cooperative Execution	Enables parallel subtask execution, improving efficiency.
Resource Sharing	Multiple processes access shared files/databases without duplication.
Computation Speedup	Parallel execution reduces total task time, increasing throughput.
Distributed Apps	IPC is essential for processes across different machines to communicate.
Information Sharing	Processes exchange data (e.g., user input, sensor readings) for joint tasks.

4.1.5 5. Conclusion

- IPC is **fundamental** for **cooperating processes** and **distributed systems**.
- It enables:
 - **Efficient task decomposition** (cooperative execution).
 - **Shared resource utilization** (avoiding redundancy).
 - **Faster computation** (parallelism).
 - **Cross-machine communication** (distributed applications).
 - **Seamless information exchange** (data propagation).
- The **three primary IPC mechanisms** (shared memory, message passing, pipes) will be explored in depth in subsequent lectures.

4.2 Device Queue

4.2.1 Introduction to Device Queue

- **Definition:** A **device queue** is a data structure used in operating systems to manage processes that are waiting to access a specific **input/output (I/O) device**.
- **Purpose:** It organizes and schedules processes that require I/O operations, ensuring orderly access to limited device resources.

4.2.2 Process State Transition and Device Queue

4.2.2.1 Waiting State and I/O Operations

- During its lifecycle, a process may transition to the **waiting state** due to:
 - The need to perform an **input/output (I/O) operation**.
 - Waiting for an **event** to occur (e.g., completion of a prior operation, signal, or resource availability).
- This lecture focuses on the **I/O operation** scenario.

4.2.2.2 Process Request for I/O Device

- When a process requires access to an **I/O device** (e.g., disk, printer, network interface), it submits a **request** to the operating system (OS).
- If the device is **available**, the OS **allocates** it to the process.
- Once allocated, the process **cannot remain in the ready queue** because:
 - The **ready queue** exclusively holds processes in the **ready state** (i.e., processes ready to execute but waiting for CPU time).
 - A process performing I/O is in the **waiting state**, not the ready state.

4.2.2.3 Removal from Ready Queue

- The process is **removed from the ready queue** and **inserted into the device queue** associated with the requested I/O device.
- This ensures the ready queue only contains processes that are **immediately eligible for CPU execution**.

4.2.3 Device Queue Structure and Functionality

4.2.3.1 Definition and Role

- Every **I/O device** in the system has an associated **device queue**.
- The device queue acts as a **holding area** for processes waiting to use the device.
- Since devices are **finite resources**, multiple processes may request the same device simultaneously.
 - Not all requests can be serviced at once; they must be **ordered and processed sequentially**.

4.2.3.2 Request Servicing

- Requests in the device queue are serviced **one after another** based on a **scheduling algorithm**.
 - The specific algorithm (e.g., First-Come-First-Served (FCFS), Shortest Seek Time First (SSTF)) is **not discussed in this lecture**.
- Once the I/O operation for a process completes:
 - The process is **removed from the device queue**.
 - It is **reinserted into the ready queue**.
 - Its state transitions from **waiting** -> **ready**.

4.2.4 Device Queue Data Structure

4.2.4.1 Implementation

- The device queue is maintained as a **linked list of Process Control Blocks (PCBs)**.
 - **Similarity to Ready Queue:** Like the ready queue, the device queue uses a linked list of PCBs to track processes.
- **Header Structure:**
 - **Head Pointer:** Points to the **first PCB** in the queue.
 - **Tail Pointer:** Points to the **last PCB** in the queue.

4.2.4.2 Example: Disk Device Queue

- **Visualization:**
 - Consider a device queue for a **hard disk**.
 - The **header** contains:
 - * **head** -> Points to the first PCB (e.g., PCB6).
 - * **tail** -> Points to the last PCB (e.g., PCB4).
 - **PCBs in Queue:**
 - * PCB6 (Process ID 6) -> PCB10 (Process ID 10) -> PCB4 (Process ID 4).
 - * The next field of PCB4 is NULL (indicating the end of the list).
 - **Ordering:**
 - * PCBs are **not necessarily ordered by Process ID (PID)**.
 - * The order depends on the **scheduling policy** (e.g., arrival time, priority).

4.2.4.3 Linked List Representation

- Each PCB contains a pointer to the **next PCB** in the queue.
- **Traversal:**
 - Start at the **head** -> Follow **next** pointers -> End at **tail** (where **next** = NULL).

4.2.5 Key Takeaways

1. **Device Queue Purpose:**
 - Manages processes waiting for I/O device access.
 - Ensures orderly and fair allocation of limited device resources.
2. **Process State Transitions:**
 - **Ready -> Waiting:** Process moves from ready queue to device queue when requesting I/O.
 - **Waiting -> Ready:** Process returns to ready queue after I/O completion.
3. **Data Structure:**
 - Implemented as a **linked list of PCBs**.
 - **Head** and **tail** pointers track the first and last PCBs, respectively.
4. **Scheduling:**
 - Requests are serviced sequentially based on an algorithm (not covered in this lecture).
5. **Example:**
 - A disk device queue may contain PCBs in any order (e.g., 6 -> 10 -> 4), not necessarily sorted by PID.

End of Notes

4.3 First process of Computer System

4.3.1 Introduction to the First Process

- The lecture focuses on identifying and understanding the **first process** created when a computer system boots.
- The discussion is centered on the **Linux operating system**, though other OS (e.g., Windows, macOS) have analogous first processes.
- The first process in Linux is critical for system initialization and subsequent process management.

4.3.2 The Init Process in Linux

4.3.2.1 Definition and Role

- The first process in Linux is called the **init process** (short for **initialization**).
- It is the **first user-space process** created by the kernel during system boot.
- **Key Characteristics:**
 - Always assigned a **Process ID (PID) of 1**, regardless of the Linux distribution.
 - Executes **continuously** while the system is powered on.
 - Terminates **only during system shutdown**.

4.3.2.2 Creation of the Init Process

- **Sequence of Events:**
 1. The **kernel** is loaded into **main memory** during boot.
 2. The kernel initializes core services, including the **init process**.
 3. The system transitions from **kernel mode** to **user mode**, where the init process begins execution.
- **Purpose:**
 - Prepares the system for **user processes** by setting up the environment (e.g., mounting filesystems, starting services).
 - Acts as the **ancestor of all other processes** (forms the root of the **process tree**).

4.3.3 Verifying the Init Process

4.3.3.1 Using the ps Command

- The `ps 1` command displays information about the init process.
 - **Output Example:**

PID	TTY	STAT	TIME	COMMAND
1	?	Ss	0:02	/sbin/init
 - **Key Fields:**
 - * **PID:** Confirms the process has PID 1.
 - * **COMMAND:** Shows the executable path (e.g., `/sbin/init`).
 - Path may vary slightly across Linux distributions (e.g., `/lib/systemd/systemd` in systems using **systemd**).

4.3.4 System Initialization: Kernel and Systemd

4.3.4.1 Kernel's Role

- During boot, the system operates in **kernel mode**.
- The kernel:
 - Loads critical drivers and modules.
 - Starts essential services, including the **systemd service manager** (or alternatives like SysVinit).

4.3.4.2 Systemd Service Manager

- **Definition:** A **software suite** responsible for system and service management in modern Linux distributions.
- **Functions:**
 - Initiates the **init process** (PID 1).
 - Manages system startup, service supervision, and logging.
- **Transition to User Mode:**
 - After kernel initialization, the system switches to **user mode**.
 - The **init process** (PID 1) becomes the first user-space process, spawning all subsequent processes.

4.3.5 Functions of the Init Process

4.3.5.1 Core Responsibilities

1. **Process Creation:**
 - Spawns **all other processes** (PID 2, 3, 4, etc.).
 - Acts as the **parent process** for orphaned processes (reparenting them via `wait()` system calls).
2. **Filesystem Management:**
 - **Mounts filesystems** (e.g., root filesystem, `/proc`, `/sys`).
 - Ensures filesystems are accessible before user processes start.
3. **System Readiness:**
 - Configures the environment for **user processes** (e.g., setting up devices, network interfaces).
 - Handles **system shutdown/reboot** gracefully.
4. **Process Tree Hierarchy:**
 - Forms the **root of the process tree**.
 - All processes trace their lineage back to PID 1 (directly or indirectly).

4.3.5.2 Interaction with System Calls

- **User Processes:**
 - When a user process invokes a **system call**, execution temporarily switches to **kernel mode**.
 - The init process itself does not handle system calls but ensures the kernel is ready to service them.

4.3.6 Process Tree and Ancestry

4.3.6.1 Hierarchical Structure

- The init process (PID 1) is the **ancestor of all processes**.
- **Visualization:**
 - A **process tree** can be viewed using tools like `ps tree` or `htop`.
 - Example:

```
init--systemd--dbus-daemon
      |         |
      |         |--networkd
      |         |
      |         |--...
      |
      |--getty
```

- **Implications:**
 - Terminating PID 1 would **crash the system** (hence it runs until shutdown).
 - Orphaned processes are **adopted by PID 1** to prevent resource leaks.

4.3.7 Variations Across Operating Systems

4.3.7.1 Linux-Specific Notes

- The **init process** is **unique to Linux/Unix-like systems**.
- Modern Linux distributions often use **systemd** as PID 1 (replacing traditional SysVinit).
 - Example: `/lib/systemd/systemd` may appear in `ps 1` output instead of `/sbin/init`.

4.3.7.2 Other Operating Systems

- **Windows:**
 - The first process is `smss.exe` (Session Manager Subsystem), which initializes the user session.
- **macOS:**
 - Uses `launchd` as the first process (PID 1), managing system services and daemons.

4.3.8 Summary of Key Points

1. **First Process in Linux:** The **init process** (PID 1), created by the kernel during boot.
2. **Permanence:** Runs until system shutdown; critical for system stability.
3. **Roles:**
 - Spawns all other processes.
 - Mounts filesystems and prepares the system for users.
 - Acts as the root of the process tree.
4. **Verification:** Use `ps 1` to confirm the init process and its executable path.
5. **Systemd:** Modern Linux systems often use **systemd** as PID 1, replacing traditional `init`.
6. **Cross-OS Differences:** Windows (`smss.exe`) and macOS (`launchd`) have analogous first processes.

4.4 Job Queue

4.4.1 Introduction to Job Queue

- The **job queue** (also called a **job pool**) is a fundamental concept in operating systems, particularly in **batch processing systems**.
- It is a **data structure** that resides in **secondary storage** (e.g., hard disk) rather than main memory (RAM).
- Its primary function is to **hold all jobs submitted by users** in a system, especially in **multi-user environments**.

4.4.2 Definition of Job Queue

- A **job queue** is a storage mechanism that:
 - **Stores jobs** submitted by one or more users.
 - **Does not execute jobs immediately**—jobs remain in the queue until they are selected for processing.
 - Is primarily used in **batch systems**, where users submit jobs for **delayed execution** rather than real-time interaction.
 - Acts as an **intermediate holding area** before jobs are loaded into **main memory** for execution.

4.4.2.1 Key Characteristics

1. **Location:**
 - Resides in **secondary storage** (e.g., hard disk, SSD).
 - Not stored in **main memory (RAM)** initially.
2. **Purpose:**
 - **Temporarily stores** jobs until the system is ready to process them.
 - Prevents **immediate execution**, allowing for **scheduled or batch processing**.
3. **Usage Context:**

- Common in **batch processing systems** (e.g., early mainframe computers, high-performance computing clusters).
- Can exist in **single-user systems** but is **more prevalent in multi-user systems**.

4.4.3 Role of Job Queue in Batch Systems

- In a **batch system**, users submit jobs **without expecting immediate results**.
- Jobs are **queued** and processed **one by one** in a **predefined order** (e.g., FIFO-First-In-First-Out, priority-based).
- Users **cannot interact** with jobs while they are in the queue or during execution.

4.4.3.1 Why Jobs Are Not Executed Immediately

- **Main memory (RAM)** has **limited capacity** and may not be able to hold all submitted jobs simultaneously.
- **Multi-user systems** receive jobs from multiple users, leading to **resource contention**.
- **Batch processing** is designed for **efficiency** rather than real-time responsiveness.

4.4.4 Need for a Job Queue

The job queue exists to address several **critical challenges** in operating systems:

4.4.4.1 1. Limited Main Memory Capacity

- **Problem:**
 - Main memory (RAM) is **finite** and may not be large enough to store all submitted jobs at once.
 - In **multi-user systems**, multiple users submit jobs **simultaneously**, exceeding available RAM.
- **Solution:**
 - Jobs are **stored in secondary storage (job queue)** instead of being rejected.
 - Jobs are **loaded into main memory only when resources are available**.

4.4.4.2 2. Resource Sharing in Multi-User Systems

- **Problem:**
 - In a **shared environment**, multiple users compete for **CPU, memory, and I/O resources**.
 - Without a job queue, some users might **monopolize resources**, leading to **unfairness**.
- **Solution:**
 - The job queue **regulates resource allocation**, ensuring **fair distribution** among users.
 - Helps maintain **system stability** by preventing resource starvation.

4.4.4.3 3. Dynamic Control of Process Load

- **Problem:**
 - The **computational load** on a system **fluctuates** over time.
 - At times, **fewer jobs** are in memory, leaving **unused capacity**.
 - At other times, **too many jobs** may be loaded, causing **performance degradation**.
- **Solution:**
 - The job queue **dynamically controls** how many jobs are loaded into main memory based on:
 - * **Available memory space.**
 - * **Current system load (CPU, I/O, etc.).**
 - Ensures **optimal resource utilization** without overloading the system.

4.4.4.4 4. Prevention of Job Rejection

- **Problem:**
 - If jobs were **only accepted when RAM is available**, users might face **rejections** during high-load periods.
- **Solution:**
 - The job queue **buffers jobs** in secondary storage, allowing them to **wait** until resources free up.
 - Improves **user experience** by ensuring jobs are **eventually processed**.

4.4.5 How the Job Queue Operates

1. **Job Submission:**
 - Users submit jobs to the system (e.g., via a command-line interface or batch script).
 - Jobs are **stored in the job queue** (secondary storage).
2. **Job Selection:**
 - The **operating system's scheduler** selects jobs from the queue based on a **scheduling algorithm** (e.g., FIFO, priority, shortest job first).
3. **Loading into Main Memory:**
 - Selected jobs are **moved from the job queue to main memory** when sufficient RAM is available.
4. **Execution:**
 - The job is **executed by the CPU**, and results are generated.
5. **Completion & Output:**
 - After execution, results are **stored or returned to the user** (e.g., in an output file or printout).

4.4.5.1 Key Observations

- Jobs in the queue are **not executable** until loaded into RAM.
- The **order of execution** depends on the **scheduling policy**.
- Users **do not interact** with jobs during execution (typical in batch systems).

4.4.6 Benefits of a Job Queue

The job queue provides several **advantages** in an operating system:

4.4.6.1 1. Efficient Memory Management

- Prevents **main memory overflow** by storing jobs in secondary storage.
- Allows the system to **handle more jobs than RAM can hold at once**.

4.4.6.2 2. Fair Resource Allocation

- Ensures **equitable distribution** of CPU, memory, and I/O among multiple users.
- Prevents **resource hogging** by any single user or job.

4.4.6.3 3. Adaptive System Performance

- Adjusts the **number of active processes** in memory based on:
 - **Available RAM.**
 - **Current system load.**
- Helps maintain **optimal performance** under varying workloads.

4.4.6.4 4. Improved User Experience

- Users can **submit jobs without immediate execution constraints**.
- Jobs are **processed eventually**, even if the system is busy initially.

4.4.6.5 5. Support for Batch Processing

- Enables **non-interactive, delayed execution** of jobs (e.g., scientific computations, payroll processing).
- Reduces the need for **real-time user intervention**.

4.4.7 Job Queue vs. Ready Queue

Feature	Job Queue	Ready Queue
Location	Secondary storage (disk)	Main memory (RAM)
State of Jobs	Not yet loaded into memory	Loaded and ready for CPU execution
Purpose	Temporary storage before execution	Holds processes waiting for the CPU
Interaction	No user interaction	May allow limited interaction
Scheduling	Long-term scheduling (job selection)	Short-term scheduling (CPU allocation)

- The **job queue** is part of **long-term scheduling** (deciding which jobs enter memory).
- The **ready queue** is part of **short-term scheduling** (deciding which process runs next on the CPU).

4.4.8 Real-World Examples of Job Queues

1. **Mainframe Computers (1960s-1980s):**
 - Users submitted **punch cards or magnetic tapes** containing jobs.
 - Jobs were **queued** and processed in batches overnight.
2. **High-Performance Computing (HPC) Clusters:**
 - Scientific simulations (e.g., weather forecasting, genetic research) are submitted to a **job scheduler** (e.g., Slurm, PBS).
 - Jobs wait in a queue until **compute nodes** are available.
3. **Print Spooling:**
 - Print jobs are **queued** in secondary storage before being sent to the printer.
 - Prevents **printer overload** and allows **ordered processing**.

4.4.9 Conclusion: Importance of Job Queue

- The **job queue** is a **critical component** in **batch processing systems** and **multi-user environments**.
- It **resolves memory limitations** by storing jobs in secondary storage.
- Ensures **fair resource sharing** and **optimal system performance**.
- Supports **delayed execution**, making it ideal for **non-interactive, compute-intensive tasks**.
- Works in conjunction with **scheduling algorithms** to manage **job selection and execution order**.

By implementing a job queue, operating systems achieve: [OK] **Better resource utilization**. [OK] **Fairness in multi-user environments**. [OK] **Scalability** for large workloads. [OK] **Reliability** in job processing.

4.5 Message Passing

4.5.1 Introduction to Message Passing

- **Definition:** Message passing is an **Inter-Process Communication (IPC)** mechanism where processes exchange data by sending and receiving messages.

- **Purpose:** Enables communication between multiple processes, either on the same system or across distributed systems.
- **Core Operations:**
 - **Send Operation:** A process transmits a message to another process.
 - **Receive Operation:** A process retrieves a message sent by another process.

4.5.2 Communication Link Establishment

- **Requirement:** Before message transmission, a **communication link** must be established between the communicating processes.
- **Function:** The link serves as the channel through which messages are transmitted.
- **Process Pairing:** For two processes (e.g., P1 and P2) to communicate, a dedicated link must exist between them.

4.5.3 Message Size Constraints

- **Fixed-Size Messages:**
 - Some systems restrict messages to a **predetermined, uniform size**.
 - Example: A system may only allow messages of 1KB.
- **Variable-Size Messages:**
 - Other systems permit messages of **varying lengths**.
 - Example: A process can send a 512-byte message followed by a 2KB message.

4.5.4 Types of Message Passing

4.5.4.1 1. Direct Message Passing

- **Definition:** The **sender or receiver process is explicitly named** in the communication operations.
- **Mechanism:**
 - The identity of the target process (recipient or sender) is **directly specified** in the **send** or **receive** operations.
 - Example:
 - * **Sending a Message:**
 - Process **P1** executes: `send(P2, message)`
 - **P2** is explicitly named as the recipient.
 - * **Receiving a Message:**
 - Process **P1** executes: `receive(P2, message)`
 - **P2** is explicitly named as the sender.
- **Key Requirement:**
 - Processes must **know the identities** (e.g., Process IDs) of the communicating parties beforehand.

4.5.4.2 2. Indirect Message Passing

- **Definition:** The **sender or receiver is not explicitly named**; communication occurs via a **mailbox**.
- **Mailbox:**
 - An **intermediary object** where messages are deposited by senders and retrieved by receivers.
 - Acts as a **communication channel** shared between processes.
 - Identified by a **system-wide unique identifier** (e.g., mailbox ID).
- **Mechanism:**
 - **Shared Mailbox:** Two processes (e.g., P1 and P2) must share a common mailbox (e.g., mailbox **x**).
 - **Sending a Message:**

- * Process **P1** executes: `send(x, message)`
- * The message is inserted into mailbox **x** (intended for P2).
- **Receiving a Message:**
 - * Process **P1** executes: `receive(x, message)`
 - * The message is retrieved from mailbox **x** (sent by P2).
- **Advantage:**
 - Processes **do not need to know each other's identities**; they only need to know the mailbox identifier.

4.5.5 Advantages of Message Passing

1. **Efficient for Small Data Transfers:**
 - Ideal for exchanging **small amounts of data** between processes.
 - Example: Sending a short command or status update.
2. **No Synchronization Required:**
 - Unlike **shared memory IPC**, message passing **does not require explicit synchronization** between processes.
 - Processes can send/receive messages **independently** without waiting for one another.
3. **Suitability for Distributed Systems:**
 - **Shared memory IPC is incompatible** with distributed systems (where processes run on different machines).
 - Message passing **naturally supports distributed environments** by transmitting messages over networks.
 - Example: Client-server communication in a distributed application.

4.5.6 Disadvantages of Message Passing

1. **Slower Than Shared Memory:**
 - **Overhead of System Calls:**
 - Each `send` and `receive` operation requires a **system call** (or API function call).
 - Example: Sending 100 messages requires **100 system calls**, each introducing latency.
 - **Performance Impact:**
 - System calls are **costly** in terms of CPU cycles and context switches.
2. **Unsuitable for Bulk Data Transfer:**
 - **Inefficient for Large Data:**
 - Transmitting large datasets (e.g., files, high-resolution images) via messages is **impractical**.
 - Example: Sending a 1GB file would require **thousands of messages**, leading to excessive overhead.
 - **Alternative:** **Shared memory** is preferred for bulk data transfer due to its **lower overhead**.

4.5.7 Comparison with Shared Memory IPC

Feature	Message Passing	Shared Memory
Synchronization	Not required	Requires explicit synchronization (e.g., semaphores)
Data Transfer Size	Best for small data	Ideal for bulk data
Distributed Systems	Supported	Not supported
Performance	Slower (due to system calls)	Faster (direct memory access)
Complexity	Simpler for small, discrete communications	More complex (requires synchronization)

4.5.8 Practical Examples

4.5.8.1 Direct Message Passing Example

- **Scenario:** Process **P1** sends a message to Process **P2**.
 - **Operation in P1:** `send(P2, "Hello, P2!")`
 - **Operation in P2:** `receive(P1, message)`
 - * P2 retrieves "Hello, P2!" from P1.

4.5.8.2 Indirect Message Passing Example

- **Scenario:** Processes **P1** and **P2** communicate via mailbox **x**.
 - **Operation in P1 (Sending):**
 - * `send(x, "Data for P2")`
 - * Message is placed in mailbox **x**.
 - **Operation in P2 (Receiving):**
 - * `receive(x, message)`
 - * P2 retrieves "Data for P2" from mailbox **x**.

4.5.9 Key Takeaways

- Message passing is a **flexible IPC mechanism** for process communication.
- **Direct message passing** requires explicit process naming, while **indirect message passing** uses mailboxes.
- **Advantages:**
 - No synchronization needed.
 - Suitable for distributed systems and small data transfers.
- **Disadvantages:**
 - Slower due to system call overhead.
 - Inefficient for bulk data transfer.
- **Use Case Selection:**
 - Choose **message passing** for small, discrete communications (e.g., signals, commands).
 - Choose **shared memory** for high-volume data transfer (e.g., large datasets).

4.6 Message Queue

4.6.1 Introduction to Message Queues

- **Definition:** A **message queue** is a data structure used in **message-passing-based interprocess communication (IPC)**.
- **Purpose:** Facilitates asynchronous communication between processes by temporarily storing messages when the receiver is unavailable.

4.6.2 Need for Message Queues

- In **message-passing IPC**, the **sender process** transmits messages to the **receiver process**.
- The receiver may not be ready to accept messages immediately due to:
 - Being busy with other tasks.
 - Not executing the receive operation at the exact moment the message is sent.
- **Solution:** Messages must be stored temporarily in a **message queue** until the receiver retrieves them.

4.6.3 Operational Mechanism of Message Queues

4.6.3.1 Basic Workflow

1. **Sender Process:**

- Inserts (appends) messages sequentially into the message queue (e.g., M0, M1, M2, ..., Mn).

2. **Receiver Process:**

- Retrieves messages from the queue at its convenience.

4.6.3.2 **Diagram Explanation**

- **Example Scenario:**

- Two processes, **P1 (sender)** and **P2 (receiver)**, communicate via a message queue.
- P1 sends messages (M0, M1, ..., Mn) into the queue.
- P2 retrieves messages from the queue in the order they were inserted (FIFO: First-In-First-Out).

4.6.3.3 **Queue Capacity and Blocking**

- **Finite Capacity:** The message queue has a fixed size (e.g., capacity = $n + 1$ messages).

- If the queue is **full** (contains $n + 1$ messages):
 - * The sender process (**P1**) **blocks** to prevent overflow.
 - * The receiver (**P2**) can continue retrieving messages to free up space.
- If the queue is **empty**:
 - * The receiver (**P2**) may:
 1. Return empty-handed (non-blocking receive).
 2. **Block** until at least one message is available (blocking receive).

4.6.4 **Message Queue Creation and Identification**

- **Creation:**

- One process (typically the **creator process**) must initialize the message queue.
- Different operating systems provide specific **function calls** (e.g., `msgget()` in Linux) to create a message queue.

- **Unique Identifier:**

- Each message queue is assigned a **system-wide unique identifier** during creation.
- Other processes must reference this identifier to access the queue for sending/receiving messages.
- **Access Control:** Without the identifier, processes cannot interact with the queue.

4.6.5 **Message Retrieval and Deletion**

- **Retrieval Behavior:**

- Once a message is **read/retrieved** by a receiver, it is **permanently deleted** from the queue.
- **Implication:** Only **one receiver** can read a message (no multiple reads).

- **Synchronous Communication:**

- Message queues enable **asynchronous communication** (receiver does not need to be active when the sender transmits).
- However, **overflow must be avoided** to prevent message loss.

4.6.6 **Message Queue Deletion**

- **Cleanup:**

- The **creator process** is responsible for deleting the message queue after communication ends (e.g., before termination).
- Ensures system resources are freed and avoids orphaned queues.

4.6.7 Advanced Feature: Multiplexing in Message Queues

- **Definition:** Allows **selective message retrieval** by multiple receiver processes based on **message types**.
- **Use Case:**
 - **Scenario:** One sender process and multiple receiver processes (e.g., Receiver 1, Receiver 2, Receiver 3).
 - **Constraint:** Each receiver should only retrieve messages of a specific **type** (e.g., Type 1, Type 2, Type 3).
- **Implementation:**
 - **Message Type Field:** Each message includes a **type attribute** set by the sender.
 - **Receiver Operation:**
 - * When executing a receive operation, the receiver specifies the **desired message type**.
 - * The operation **blocks** until a message of the specified type is available.
- **Example:**
 - Sender inserts messages with types: Type 1, Type 2, Type 3.
 - Receiver 1 requests Type 1 -> retrieves only Type 1 messages.
 - Receiver 2 requests Type 2 -> retrieves only Type 2 messages.
- **Key Point:** Enables **targeted communication** without broadcasting to all receivers.

4.6.8 Summary of Key Concepts

1. **Message Queue:** A temporary storage data structure for messages in IPC.
2. **Asynchronous Communication:** Sender and receiver do not need to be active simultaneously.
3. **FIFO Order:** Messages are retrieved in the order they were sent.
4. **Blocking Conditions:**
 - Sender blocks if the queue is full.
 - Receiver blocks if the queue is empty (or until a specific message type arrives).
5. **Unique Identifier:** Required for processes to access the queue.
6. **Message Deletion:** Messages are removed upon retrieval.
7. **Multiplexing:** Supports multiple receivers filtering messages by type.
8. **Resource Management:** Creator process must delete the queue post-communication.

4.6.9 Practical Implications

- **Avoiding Overflow:** Critical to prevent message loss; sender must handle blocking gracefully.
- **Efficiency:** Enables decoupled communication, improving system flexibility.
- **Linux Environment:** Commonly used in Unix-like systems with robust support for message types and multiplexing.

4.7 Pipe

4.7.1 Introduction to Pipes

- **Definition:** A **pipe** is an **Inter-Process Communication (IPC)** mechanism that allows processes to exchange data.
- **Function:** Acts as a **communication channel** between processes.
 - One process **writes** data into the pipe (sender/producer).
 - Another process **reads** data from the pipe (receiver/consumer).
- **Data Handling:**
 - Information passed through a pipe is **not treated as distinct messages** (unlike message queues).
 - Data flows as a **continuous stream** of bytes.

4.7.2 Types of Pipes

Two primary types of pipes exist for IPC: 1. **Ordinary Pipes (Anonymous Pipes)** 2. **Named Pipes (FIFOs)**

4.7.3 Ordinary Pipes (Anonymous Pipes)

4.7.3.1 Characteristics

- **Unidirectional Communication:**
 - Data flows **only in one direction** (from writer to reader).
 - Reverse communication requires **two separate pipes**.
- **Process Relationship Requirement:**
 - **Only accessible between parent and child processes** (must share a parent-child relationship).
 - Unrelated processes **cannot** use ordinary pipes for communication.
- **Lifetime:**
 - Ceases to exist when **all processes using it terminate**.
 - On **Windows**, ordinary pipes are called **anonymous pipes**.

4.7.3.2 Structure of an Ordinary Pipe

- Consists of **two ends**:
 1. **Read End** - Used by the **reader/consumer** process.
 2. **Write End** - Used by the **writer/producer** process.
- **File Descriptor Representation (Linux):**
 - Both ends are represented as **file descriptors**.
 - Inherited by child processes (since pipes are treated as files).

4.7.3.3 Communication Between Parent and Child Processes

1. **Pipe Creation:**
 - Parent process **creates the pipe** before forking a child.
 - Child process **inherits** the pipe (including file descriptors).
2. **Role Assignment:**
 - **Parent as Writer, Child as Reader:**
 - Parent writes to the **write end**.
 - Child reads from the **read end**.
 - **Roles Can Be Reversed:**
 - Child can write, and parent can read.
3. **Best Practices:**
 - **Close Unused Ends:**
 - **Reader process** should **close the write end** (not needed).
 - **Writer process** should **close the read end** (not needed).
 - Prevents **resource leaks** and **deadlocks**.

4.7.3.4 Example Scenario (Parent-Child Communication)

- **Parent Process:**
 - Creates pipe -> forks child -> writes data to pipe -> closes read end.
- **Child Process:**
 - Reads data from pipe -> closes write end.
- **Termination:**
 - Pipe is **automatically destroyed** when both processes terminate.

4.7.4 Named Pipes (FIFOs)

4.7.4.1 Characteristics

- **Bidirectional Communication:**
 - Supports **two-way data flow** (unlike ordinary pipes).
- **No Parent-Child Relationship Required:**
 - **Any processes** (related or unrelated) can communicate.
- **Persistence:**
 - **Continues to exist** even after communicating processes terminate.
 - Must be **explicitly deleted** (unlike ordinary pipes).
- **Multiple Writers/Readers:**
 - A single named pipe can be used by **multiple processes** (writers/readers).
- **Linux Terminology:**
 - Named pipes are called **FIFOs (First-In-First-Out)**.

4.7.4.2 Use Cases

- **Server-Client Models:**
 - Clients write requests to a named pipe; server reads and responds.
- **Logging Systems:**
 - Multiple processes log data to a central named pipe.
- **Inter-Process Data Sharing:**
 - Unrelated processes exchange data without shared memory.

4.7.4.3 Comparison with Ordinary Pipes

Feature	Ordinary Pipe (Anonymous)	Named Pipe (FIFO)
Direction	Unidirectional	Bidirectional
Process Relationship	Parent-Child only	Any processes
Persistence	Dies with processes	Persists after process termination
Multiple Users	No (1 writer, 1 reader)	Yes (multiple writers/readers)
OS Terminology	Anonymous Pipe (Windows)	FIFO (Linux)

4.7.5 Key Takeaways

1. **Pipes as IPC Mechanism:**
 - Enable **secure, efficient** data exchange between processes.
2. **Ordinary Pipes:**
 - **Unidirectional, parent-child only, short-lived.**
 - Requires **explicit closing of unused ends.**
3. **Named Pipes (FIFOs):**
 - **Bidirectional, persistent, no relationship constraints.**
 - Useful for **complex IPC scenarios** (e.g., client-server models).
4. **Linux Implementation:**
 - Pipes are **file-descriptor-based** (inherited by child processes).
 - Named pipes appear as **special files** in the filesystem.
5. **Best Practices:**
 - Always **close unused pipe ends** to avoid leaks.
 - For bidirectional communication, **named pipes are preferred.**

4.7.6 Conclusion

Pipes are a fundamental **IPC mechanism** in operating systems, enabling processes to communicate **without shared memory**. While **ordinary pipes** are simple and restricted to parent-child processes, **named pipes (FIFOs)** offer **greater flexibility** with bidirectional communication and persistence. Understanding their **structure, limitations, and use cases** is crucial for designing efficient inter-process communication systems.

4.8 Process Context Switch

4.8.1 Introduction to Process Context Switch

- **Definition:** A **process context switch** refers to the mechanism by which the CPU pauses the execution of one process (the **old process**) and begins executing another process (the **new process**).
 - The **old process** is not necessarily terminated; it may have been halted temporarily (e.g., due to an interrupt, system call, or time slice expiration).
 - The **new process** may not be executing for the first time; it could be resuming from a prior halt.
- **Key Objective:**
 - Ensure the **old process** can **resume** (not restart) execution later from the exact point where it was interrupted.
 - Load the state of the **new process** to begin or resume its execution.

4.8.2 Why Context Switching is Necessary

- **State Preservation:**
 - The **old process** is halted but not terminated, so its **state** must be saved to allow later resumption.
 - Without saving the state, the process would have to restart from scratch, leading to loss of progress and incorrect behavior.
- **State Loading for New Process:**
 - The **new process** requires its previously saved state to be loaded to continue execution from where it left off (if it was interrupted earlier).
- **Role of the Process Control Block (PCB):**
 - The **PCB** stores the **context** (state) of a process, which includes:
 - * **CPU register contents:** Temporary values or data the process was manipulating.
 - * **Program Counter (PC):** Address of the next instruction to be executed (critical for resumption).
 - * **Process state:** Not just the high-level states (e.g., *new*, *ready*, *running*), but the **detailed operational state** of the process.
 - * **Memory management information:**
 - Allocated memory addresses or ranges.
 - Memory usage statistics (e.g., page tables, segment tables).

4.8.3 Context Switch Overhead

- **Definition:** The **context switch time** is the duration during which the CPU is **not executing any process** but is instead:
 - Saving the state of the **old process**.
 - Loading the state of the **new process**.
- **Characteristics:**
 - **Pure overhead:** No useful computation occurs during this time.
 - **Hardware-dependent:** The speed of context switching depends on the system's hardware support (e.g., register save/restore efficiency, memory access speed).
 - **Performance impact:**
 - * High context switch times degrade system performance, especially in **multitasking environments**.

- * In multitasking, frequent context switches are necessary to maintain the illusion of parallel execution. If switches are slow, the system appears sluggish to users.

4.8.4 Steps in a Process Context Switch

The following steps outline the flow of a context switch between two processes, **P1** (old process) and **P2** (new process), with the involvement of the **Operating System (OS)**.

4.8.4.1 1. Interruption of the Old Process (P1)

- **Trigger:** P1 receives an **interrupt** or invokes a **system call**.
 - Examples of interrupts:
 - * **Timer interrupt:** Time slice (quantum) expires.
 - * **I/O interrupt:** P1 requests an I/O operation and is blocked.
 - * **System call:** P1 requests a service from the OS (e.g., file access).
- **Result:** Execution of P1 is **halted**, and P1 transitions to an **idle** or **blocked** state.

4.8.4.2 2. Saving the State of P1

- The OS saves the **context** of P1 into its **PCB (PCB1)**.
 - **Saved components:**
 - * CPU registers (general-purpose, stack pointer, etc.).
 - * Program Counter (PC).
 - * Process state (e.g., *ready*, *blocked*).
 - * Memory management data (e.g., page tables).
- **Storage location:** The PCB may reside in the **process's stack** or an **OS-defined data structure**.

4.8.4.3 3. Selecting the New Process (P2)

- The OS's **scheduler** selects the next process to execute (P2).
- P2 may be:
 - A **newly created process** (first execution).
 - A **previously interrupted process** (resuming execution).

4.8.4.4 4. Loading the State of P2

- The OS retrieves the **context** of P2 from its **PCB (PCB2)**.
 - **Loaded components:**
 - * CPU registers (restored to their values when P2 was last interrupted).
 - * Program Counter (set to the next instruction to execute).
 - * Memory management data (e.g., reloading page tables).
- **Note:** If P2 was interrupted earlier, this step is a **reload** of its saved state.

4.8.4.5 5. Execution of P2

- The CPU begins executing P2 from the instruction pointed to by its **Program Counter**.
- P2 runs until it is **interrupted** or **completes its time slice**.

4.8.4.6 6. Interruption of P2 (Cycle Repeats)

- P2 may eventually be interrupted (e.g., by a timer or system call).
- The OS saves P2's state into **PCB2** and selects another process (e.g., P1 if its system call completes).

4.8.4.7 7. Resuming P1

- If P1's pending operation (e.g., I/O or system call) completes, the OS:
 - Reloads P1's state from **PCB1**.
 - Resumes P1's execution from the **exact instruction** where it was interrupted.

4.8.5 Example Walkthrough

Consider two processes, **P1** and **P2**, and the following sequence:

1. **Initial State:**
 - P1 is **running**; P2 is **idle** (ready to run).
 - P1's state is actively stored in CPU registers.
2. **Interrupt Occurs for P1:**
 - P1 is interrupted (e.g., timer expires).
 - OS saves P1's state to **PCB1**.
3. **Context Switch to P2:**
 - OS loads P2's state from **PCB2** into the CPU.
 - P2 begins execution.
4. **Interrupt Occurs for P2:**
 - P2 is interrupted (e.g., system call).
 - OS saves P2's state to **PCB2**.
5. **Resuming P1:**
 - P1's pending operation completes (e.g., I/O finishes).
 - OS reloads P1's state from **PCB1**.
 - P1 resumes execution from where it left off.

4.8.6 Key Observations

- **Context Switch Frequency:**
 - In **multitasking systems**, context switches occur **frequently** to share CPU time among processes.
 - Efficient context switching is critical to maintain **responsiveness** and **performance**.
- **Hardware Influence:**
 - Faster hardware (e.g., quicker register saves/restores) reduces context switch overhead.
 - Modern CPUs include **dedicated instructions** (e.g., **task switch** in x86) to optimize context switches.
- **PCB Storage:**
 - The PCB can be stored in:
 - * The **process's kernel stack**.
 - * A **dedicated OS data structure** (e.g., a linked list of PCBs).

4.8.7 Summary of Key Concepts

1. **Context Switch:** The process of switching the CPU from one process to another, involving saving the old process's state and loading the new process's state.
2. **PCB (Process Control Block):** Stores the **context** of a process, including registers, PC, and memory management data.
3. **Overhead:** Context switch time is **non-productive** and should be minimized for efficiency.
4. **Multitasking Dependency:** Frequent context switches are essential for multitasking but must be fast to avoid performance degradation.
5. **Resumption vs. Restart:** Processes **resume** from their interruption point, not restart from the beginning.

4.9 Process Control Block (PCB)

4.9.1 1. Introduction to Process Identification

- Every **process** in an operating system (OS) must be uniquely identified and managed.
- The OS maintains a **one-to-one relationship** between each process and its **Process Control Block (PCB)**.
- The primary mechanism for identifying a process is the **Process Identifier (PID)**.

4.9.1.1 1.1 Process Identifier (PID)

- A **PID** is a **system-wide unique integer** assigned to each process.
- PIDs are **monotonically increasing** (each new process gets a higher PID than the previous one).
- **No two active processes** in the system can have the same PID.
- Example: On a **Linux system**, the command `ps -el` displays all active processes along with their PIDs (found in the **fourth column** of the output).

4.9.2 2. Process Representation in the Operating System

- A process is represented in the OS through a **data structure** called the **Process Control Block (PCB)**.
 - Also referred to as a **Task Control Block (TCB)** (since “process” and “task” are used interchangeably).
- **Key Property:** There is a **one-to-one mapping** between a process and its PCB.
 - Every process has **exactly one PCB**, and every PCB corresponds to **exactly one process**.

4.9.3 3. Structure of the Process Control Block (PCB)

The PCB stores **all essential information** about a process, categorized into several key components:

4.9.3.1 3.1 Process State

- Indicates the **current state** of the process in its **lifecycle**.
- Possible states (from process lifecycle models):
 - **New** (being created)
 - **Ready** (waiting for CPU allocation)
 - **Running** (currently executing)
 - **Waiting/Blocked** (waiting for an I/O operation or event)
 - **Terminated** (completed execution)

4.9.3.2 3.2 Program Counter (PC) Value

- Stores the **address of the next instruction** to be executed for the process.
- Critical for **context switching** (when the OS switches between processes).

4.9.3.3 3.3 CPU Registers

- Contains the **contents of all CPU registers** associated with the process.
- Register types vary by **computer architecture**, but common ones include:
 - **Accumulator** (used for arithmetic/logic operations)
 - **Index Register** (used for addressing)
 - **General-Purpose Registers** (for temporary data storage)
- These registers must be **saved and restored** during context switches.

4.9.3.4 3.4 CPU Scheduling Information

- Contains data that influences **how the process is allocated CPU time**.
 - **Process Priority**: Determines scheduling order (higher priority may get CPU first).
 - **Pointers to Scheduling Queues**: Links to data structures (e.g., ready queue, wait queue) where the process resides while waiting for CPU allocation.
 - **Other Scheduling Parameters**: May include:
 - * Time quantum (for round-robin scheduling)
 - * Recent CPU usage (for priority adjustments)

4.9.3.5 3.5 Memory Management Information

- Tracks **memory allocation** and usage for the process.
 - **Memory Allocation Details**:
 - * Base and limit registers (for contiguous memory allocation).
 - * Memory range (start and end addresses) assigned to the process.
 - **Memory Management Scheme-Specific Data**:
 - * **Page Tables** (for **paging-based** memory management).
 - * **Segment Tables** (for **segmentation-based** memory management).

4.9.3.6 3.6 Accounting Information

- Used for **resource tracking, billing, and performance monitoring**.
 - **CPU Utilization**: Percentage of CPU time used by the process.
 - **Execution Time**: Total time the process has been running.
 - **Time Limits**: Maximum allowed execution time (user-defined or system-imposed).
 - **Process Number (PID)**: Unique identifier for the process.
 - **User Account Information**: Identifies the user who created the process (for permissions and billing).

4.9.3.7 3.7 Input/Output (I/O) Status Information

- Tracks **I/O resources** allocated to or used by the process.
 - **List of Allocated I/O Devices**: Printers, disks, network interfaces, etc.
 - **Open File Descriptors**: Files currently in use by the process.
 - **I/O Operation Status**: Whether an I/O request is pending, completed, or failed.

4.9.4 4. Summary of PCB Functionality

The PCB serves as the **comprehensive representation** of a process in the OS, storing: - **Process State** (lifecycle stage) - **Execution Context** (Program Counter, CPU registers) - **Resource Allocation** (CPU scheduling, memory, I/O) - **Accounting & Monitoring Data** (CPU usage, execution time, user info) - **Process Identity** (PID, user account)

4.9.4.1 Key Takeaways:

1. **PID** is a **unique, system-wide integer** that identifies a process.
2. **PCB** is a **mandatory data structure** for every process, maintaining all necessary process-related information.
3. The OS uses the PCB for:
 - **Process scheduling** (deciding which process runs next).
 - **Context switching** (saving/restoring process state).
 - **Resource management** (memory, I/O, CPU allocation).

- **Accounting & security** (tracking usage, enforcing permissions).

4.9.5 5. Conclusion

- The **Process Control Block (PCB)** is the **central data structure** that enables the OS to manage processes efficiently.
- It ensures that the OS can:
 - **Identify** processes uniquely (via PID).
 - **Track** their state and resource usage.
 - **Switch** between processes seamlessly (via context saving).
 - **Enforce** scheduling and memory management policies.
- Understanding the PCB is **fundamental** to grasping how modern operating systems handle **multitasking, resource allocation, and process management**.

4.10 Process Creation

4.10.1 Introduction to Process Creation in Linux

- Process creation in Linux is achieved using the **fork** system call.
- **fork** creates a new process by duplicating the calling (parent) process.
- The new process is called the **child process**, while the original is the **parent process**.
- Both processes continue execution independently after the **fork** call.

4.10.2 The fork System Call

4.10.2.1 Definition and Purpose

- **fork** is a function call used to create a new process in Linux.
- When a process calls **fork**, the operating system creates an **exact copy** of the calling process.
- The child process inherits:
 - The **program code** (instructions) of the parent.
 - All **data variables** and **data structures** (e.g., global variables, heap memory, stack).
 - The **execution context** (e.g., program counter, registers).

4.10.2.2 Key Characteristics

- **Address Space Separation:**
 - The child process receives a **copy** of the parent's address space.
 - **No shared memory:** Changes to variables in the parent **do not** affect the child, and vice versa.
 - * Example: If the parent modifies a variable **x**, the child's **x** remains unchanged.
- **Execution Flow:**
 - After **fork**, **both processes resume execution** from the instruction **immediately following** the **fork** call.
 - The child does **not** restart the program from the beginning.

4.10.2.3 Return Values

- **fork** returns different values to the parent and child:
 - **Parent Process:** Receives the **PID (Process ID)** of the newly created child.
 - **Child Process:** Receives **0**.
 - **Error Case:** Returns **-1** if the fork fails (e.g., due to resource limitations).

4.10.2.4 Function Signature and Header File

- **Signature:**

```
pid_t fork(void);
```

- **Return Type:** `pid_t` (a POSIX-standard data type for process IDs).
- **Arguments:** None (`void`).

- **Header File:**

- Defined in `unistd.h` (must be included in the source code).
- The `pid_t` type is defined in `sys/types.h` (also required for compatibility).

4.10.3 The `pid_t` Data Type

- **Purpose:** Represents **Process IDs (PIDs)** in Linux/Unix systems.
- **Nature:**
 - Internally, `pid_t` is typically an **integer** (e.g., `int` or `long`).
 - Standardized by **POSIX** to ensure portability across Unix-like systems.
- **Usage:**
 - Used for storing and manipulating PIDs in system calls (e.g., `fork`, `getpid`).
 - Example declaration:

```
pid_t child_pid = fork();
```

4.10.4 Example: Basic Process Creation

```
#include <unistd.h>
#include <stdio.h>

int main() {
    fork();           // Create a child process
    printf("Hello\n"); // Both parent and child execute this
    return 0;
}
```

4.10.4.1 Code Snippet

4.10.4.2 Execution Flow

1. The program starts as a single process (e.g., `a.out` with PID 1101).
2. When `fork()` is called:
 - The parent (PID 1101) continues execution.
 - A child process is created (e.g., PID 1102).
3. **Both processes** execute the `printf("Hello\n")` statement.
 - **Output:** Two lines of "Hello" are printed (one by parent, one by child).
 - **Limitation:** The output does not distinguish which process printed each line.

4.10.4.3 Visualization

Parent Process (P1, PID=1101)

|

```

    |-- fork() --+
    |             |
Child Process (C, PID=1102)

```

- After fork, **both P1 and C execute independently** (concurrently if the system supports multiprocessing).

4.10.5 Accessing Process IDs Programmatically

4.10.5.1 getpid: Retrieve the Current Process ID

- **Purpose:** Returns the **PID of the calling process**.
- **Signature:**

```
pid_t getpid(void);
```

- **Return Type:** pid_t (PID of the current process).
- **Arguments:** None.

- **Header File:** Defined in unistd.h.
- **Example Usage:**

```
pid_t my_pid = getpid();
printf("My PID: %d\n", my_pid);
```

4.10.5.2 getppid: Retrieve the Parent Process ID

- **Purpose:** Returns the **PID of the parent process (PPID)**.
- **Signature:**

```
pid_t getppid(void);
```

- **Return Type:** pid_t (PPID of the current process).
- **Arguments:** None.

- **Header File:** Defined in unistd.h.
- **Example Usage:**

```
pid_t parent_pid = getppid();
printf("Parent PID: %d\n", parent_pid);
```

```

#include <unistd.h>
#include <stdio.h>

int main() {
    pid_t pid = fork(); // Create child process

    if (pid == 0) {
        // Child process
        printf("Child PID: %d, Parent PID: %d\n", getpid(), getppid());
    } else {
        // Parent process
    }
}

```

```

        printf("Parent PID: %d, Child PID: %d\n", getpid(), pid);
    }
    return 0;
}

```

4.10.5.3 Practical Example with fork, getpid, and getppid

4.10.5.3.1 Expected Output:

Parent PID: 1101, Child PID: 1102

Child PID: 1102, Parent PID: 1101

- The parent prints its PID (1101) and the child's PID (1102).
- The child prints its PID (1102) and its parent's PID (1101).

4.10.6 Key Takeaways

1. Process Creation:

- Use `fork()` to duplicate a process.
- The child is an **independent copy** of the parent (no shared memory by default).

2. Return Values:

- Parent gets the **child's PID**.
- Child gets `0`.

3. Execution Flow:

- Both processes continue from the line after `fork()`.

4. Process IDs:

- Use `getpid()` to get the current process's PID.
- Use `getppid()` to get the parent's PID.

5. Header Files:

- Include `unistd.h` for `fork`, `getpid`, and `getppid`.
- Include `sys/types.h` for the `pid_t` type.

4.11 Process Termination

4.11.1 Introduction to Process Termination

- **Definition:** Process termination occurs when a process completes the execution of its final statement and exits.
- **Mechanism:**
 - A process invokes the `exit` function call either **directly** or **indirectly** (e.g., via a `return` statement).
 - Upon termination, the process returns an **exit status** to its **parent process**, provided:
 - * The parent has called `wait` for the child process.
 - * The parent has passed a **normal argument** to the `wait` function.
 - **Exit Status Values:**
 - * **Normal termination:** Exit value = `0` (returned to parent).
 - * **Abnormal termination:** Exit value = **non-zero** (returned to parent).
 - **Resource Reclamation:** All resources allocated to the terminated process are released and reclaimed by the **operating system**.

4.11.2 Termination via External Commands

- **kill Command:**

- A process (typically a parent) can terminate another process (typically a child) using the `kill` command.
- Reasons for parent-initiated termination:
 1. **Parent Termination Constraint:**
 - * The operating system may **not allow child processes to exist** if the parent terminates.
 - * Example: If the parent process must exit, all its children are forcibly terminated.
 2. **Task Irrelevance:**
 - * The task assigned to the child process is **no longer required** or relevant.
 - * Example: A child process spawned to perform a background task that is canceled mid-execution.

4.11.3 Operating System Policies on Process Termination

- **Strict Parent-Child Dependency:**
 - Some operating systems **automatically terminate all child processes** if the parent process terminates.
 - Example: A **tree of processes** (parent → children → grandchildren) will collapse if the root parent terminates.
- **Cascading Termination:**
 - **Definition:** A phenomenon where the termination of a parent process triggers the termination of **all its descendant processes** (children, grandchildren, etc.).
 - **Example Scenario:**
 - * Process **P** (parent) creates **C1** (child).
 - * **P** creates another child, **C2**.
 - * **C1** creates a child, **C3** (grandchild of **P**).
 - * If **P** terminates, **C1**, **C2**, and **C3** are **automatically terminated**.
 - **Exception:**
 - * **Linux (e.g., Ubuntu):** Does **not** enforce cascading termination by default. Child processes may continue executing even if the parent terminates.

4.11.4 Types of Processes in Process Termination

4.11.4.1 1. Zombie Processes

- **Definition:**
 - A **zombie process** (or **defunct process**) is a process that has **terminated** but whose entry **remains in the process table** because the parent process has **not yet called wait** to collect its exit status.
- **Why Zombie Processes Exist:**
 - The operating system retains the process entry to allow the parent to **retrieve the exit status** later.
 - The zombie state persists until the parent invokes `wait`.
- **Characteristics:**
 - The process is **not active** (no execution or resource consumption).
 - The process table retains the **PID** and **exit status**.
 - **Every terminating process becomes a zombie briefly** (until `wait` is called).
- **Removal from Process Table:**
 - Occurs when the parent calls `wait`, which:
 - * Collects the exit status.
 - * Removes the zombie's entry from the process table.
- **Identifying Zombie Processes:**
 - **Command 1:**

```
ps -el | grep 'Z'
```

- * `ps -el`: Lists all processes with detailed information.

- * `grep 'Z'`: Filters for processes with state **'Z'** (zombie).
- **Command 2:**

```
ps -el | grep 'defunct'
```

- * `defunct`: Alternative keyword for zombie processes.
- **Output Interpretation:**
 - * The **'S'** column in `ps` output shows **'Z'** for zombie processes.
 - * Example output fields: **PID**, **PPID (parent PID)**, **state (Z)**, etc.

4.11.4.2 2. Orphan Processes

- **Definition:**
 - An **orphan process** is a child process whose **parent process has terminated** before the child completes execution.
- **Behavior of Orphan Processes:**
 - The child continues execution **without a parent**.
 - Upon termination, the orphan **becomes a zombie** (since no parent exists to call `wait`).
- **Role of the init Process:**
 - **init Process:** The first process created by the system (PID = 1 in Linux).
 - **Adoption Mechanism:**
 - * The operating system **reassigns orphan processes** to `init` as their new parent.
 - * `init` periodically calls `wait` to:
 - Collect the exit status of adopted zombies.
 - Remove their entries from the process table.
 - **Purpose:** Prevents accumulation of zombie processes in the system.

4.11.5 Summary of Key Concepts

1. **Process Termination:**
 - Triggered by `exit` or `return` statements.
 - Returns an exit status to the parent (if `wait` is called).
 - Releases all allocated resources.
2. **Parent-Initiated Termination:**
 - Parents may terminate children via `kill` (e.g., due to irrelevance or system constraints).
3. **Cascading Termination:**
 - Parent termination may forcefully terminate all descendants (OS-dependent).
4. **Zombie Processes:**
 - Terminated processes with unresolved exit statuses (parent hasn't called `wait`).
 - Identifiable via `ps -el | grep 'Z'` or `ps -el | grep 'defunct'`.
5. **Orphan Processes:**
 - Child processes whose parents terminate prematurely.
 - Adopted by `init`, which handles their termination and exit status collection.

4.12 Putting it all together

4.12.1 Introduction

This lecture focuses on integrating `fork()`, `exec()`, and `wait()` system calls in Unix/Linux process management. The goal is to demonstrate how these functions work together to create, replace, and synchronize processes.

4.12.2 Key System Calls Review

4.12.2.1 1. fork()

- **Purpose:** Creates a new process (child) as a copy of the calling process (parent).
- **Return Values:**
 - **In the parent:** Returns the **PID of the child** (a positive integer).
 - **In the child:** Returns **0**.
 - **On failure:** Returns **-1** (no child process created).
- **Data Type:** Uses `pid_t` (defined in `<sys/types.h>`).

4.12.2.2 2. exec() Family (e.g., `execvp()`)

- **Purpose:** Replaces the current process image with a new program.
- **Behavior:**
 - If successful, the original process image is **completely overwritten** (no return to the calling code).
 - If unsuccessful, returns **-1** (original process continues).
- **Variants:**
 - `execvp()`: Searches for the executable in the PATH environment variable.
 - * **Arguments:**
 1. Path to the executable (or just the command name if in PATH).
 2. Command-line arguments (including `argv[0]`, typically the program name).
 3. Must end with `NULL`.

4.12.2.3 3. wait()

- **Purpose:** Causes the parent process to **pause** until a child process terminates.
- **Behavior:**
 - Blocks the parent until the child exits.
 - Returns the **PID of the terminated child** or **-1** on error.
 - Used to **synchronize** parent and child processes.

4.12.3 Programming Example 1: Executing `ls -l` in a Child Process

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t p = fork(); // Create a child process

    if (p == 0) { // Child process
        execvp("/bin/ls", "ls", "-l", NULL);
        printf("Hello\n"); // This line is never reached
    }
    else { // Parent process
        wait(NULL); // Wait for child to finish
        printf("Bye\n");
    }
}
```

```
    return 0;
}
```

4.12.3.1 Code Overview

4.12.3.2 Key Observations

1. Child Process ($p == 0$):

- Calls `execvp("/bin/ls", "ls", "-l", NULL)` to replace its image with the `ls` command.
- **Arguments:**
 - `"/bin/ls"`: Path to the `ls` executable.
 - `"ls"`: `argv[0]` (program name).
 - `"-l"`: Option for long listing format.
 - `NULL`: Terminates the argument list.
- `printf("Hello\n")` is **never executed** because `execvp()` overwrites the child's memory if successful.

2. Parent Process ($p > 0$):

- Calls `wait(NULL)` to pause until the child terminates.
- Prints `"Bye"` **only after** the child completes.

4.12.3.3 Output Analysis

- **Order of Execution:**
 1. Child executes `ls -l` -> prints directory contents.
 2. Parent waits -> prints `"Bye"` after child finishes.
- **"Hello" is missing:** Confirms that `execvp()` does not return on success.

4.12.4 Programming Example 2: Chaining `exec()` Calls

4.12.4.1 Code Overview

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t p = fork();

    if (p == 0) { // Child process
        printf("Child PID = %d\n", getpid());
        printf("Parent PID of child = %d\n", getppid());
        execvp("./sample2.out", "./sample2.out", NULL);
    }
    else { // Parent process
        wait(NULL);
        printf("Parent PID = %d\n", getpid());
        printf("Bye\n");
    }
}
```

```

    }
    return 0;
}

```

4.12.4.1.1 File: sample1.c (Parent Program)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    execlp("/bin/cat", "cat", "MyFile", NULL);
    return 0;
}

```

4.12.4.1.2 File: sample2.c (Child Program)

4.12.4.2 Key Observations

1. **Child Process (sample1.c):**
 - Prints its **PID** (getpid()) and **parent's PID** (getppid()).
 - Calls execlp("./sample2.out", ...) to replace its image with sample2.out.
2. **sample2.out (Replacement Program):**
 - Executes execlp("/bin/cat", "cat", "MyFile", NULL) to display the contents of MyFile.
 - **Arguments:**
 - "/bin/cat": Path to the cat executable.
 - "cat": argv[0].
 - "MyFile": File to display.
 - NULL: Terminates the argument list.
3. **Parent Process (sample1.c):**
 - Waits for the child (sample2.out) to finish.
 - Prints its PID and "Bye".

4.12.4.3 Output Analysis

1. **Child Output:**
 - Prints its PID and parent's PID.
 - Executes sample2.out -> runs cat MyFile -> displays file contents.
2. **Parent Output:**
 - Prints its PID (matches the parent PID printed by the child).
 - Prints "Bye" after the child terminates.

4.12.4.4 Verification

- **Parent-Child PID Consistency:**
 - The parent PID printed by the child (getppid()) matches the parent's own PID (getpid()).
- **Process Replacement:**
 - sample2.out replaces the child's image, and cat replaces sample2.out's image.

4.12.5 Key Takeaways

1. **fork() + exec() Pattern:**
 - **fork()** creates a child process.
 - **exec()** replaces the child's image with a new program.
 - The parent can use **wait()** to synchronize execution.
2. **exec() Behavior:**
 - **Successful call:** The original process image is **lost forever** (no return).
 - **Failed call:** Returns **-1**, and the original process continues.
3. **Synchronization with wait():**
 - Ensures the parent waits for the child to complete before proceeding.
 - Prevents **zombie processes** (terminated children not reaped by the parent).
4. **Debugging Tips:**
 - If a **printf()** after **exec()** executes, the **exec()** call **failed**.
 - Always check return values of **fork()** and **exec()**.

4.12.6 Common Pitfalls

1. **Missing NULL in exec():**
 - Omitting NULL at the end of **exec()** arguments causes undefined behavior.
2. **Incorrect Paths in exec():**
 - Ensure the executable path is correct (use absolute paths or PATH-resolved names).
3. **Ignoring wait():**
 - Without **wait()**, the parent may terminate before the child, leading to zombies.
4. **Assuming exec() Returns:**
 - Code after **exec()** in the child is **unreachable** if **exec()** succeeds.

4.13 Ready Queue

4.13.1 Introduction to the Ready Queue

- The **ready queue** is a **data structure** that resides in **main memory** (RAM).
- It holds a **subset of jobs** (or processes) selected from the **job queue**.
 - The **job queue** is a data structure that contains **all submitted jobs** in the system and resides on **secondary storage** (e.g., disk).
 - Only a portion of these jobs are loaded into **main memory** for execution.

4.13.1.1 Purpose of the Ready Queue

- The ready queue **stores processes that are in the ready state**.
 - A process in the **ready state** is **waiting for CPU allocation** to execute.
 - These processes have all necessary resources (except the CPU) to run.
- The ready queue **manages the order** in which processes compete for CPU time.

4.13.2 Relationship Between Ready Queue and Degree of Multiprogramming

- The **degree of multiprogramming** is defined as: *> The number of jobs or processes present in the ready queue at any given time.*
- **Key Implications:**
 - If the ready queue contains **10 processes**, and there are **10 available CPU cores**, all 10 processes can run **simultaneously** (assuming no other bottlenecks).

- If there are **fewer CPU cores than processes** in the ready queue, only a subset of processes will execute at a time (determined by the **CPU scheduler**).
- A higher degree of multiprogramming **increases system throughput** by keeping the CPU busy, but excessive processes may lead to **thrashing** (excessive context switching).

4.13.3 Data Structure of the Ready Queue

- The ready queue is implemented as a **linked list of Process Control Blocks (PCBs)**.
 - Each **node** in the linked list represents a **PCB** (one process).
 - The linked list is **dynamically updated** as processes enter (enqueue) or leave (dequeue) the ready state.

4.13.3.1 Components of the Ready Queue Linked List

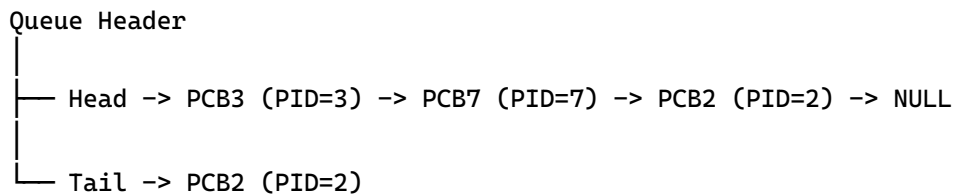
1. Queue Header (Sentinel Node)

- Acts as the **entry point** to the ready queue.
- Contains **two pointers**:
 - **Head pointer**: Points to the **first PCB** in the queue.
 - **Tail pointer**: Points to the **last PCB** in the queue.
- The header itself **does not represent a process**; it is purely an organizational node.

2. Process Control Blocks (PCBs)

- Each PCB contains:
 - Process metadata (e.g., **Process ID (PID)**, state, program counter, registers).
 - A **next pointer** to link to the subsequent PCB in the queue.
- The **last PCB** in the queue has its **next** pointer set to **NULL**, indicating the end of the list.

4.13.3.2 Example Structure Consider a ready queue with **three processes** (PID: 3, 7, 2):



- **Ordering:**
 - Processes are **not stored in any mandatory order** (e.g., PID sequence).
 - The example shows **PCB3 -> PCB7 -> PCB2**, demonstrating arbitrary ordering.
 - The **scheduler** determines the execution order (e.g., FIFO, priority-based).

4.13.4 Operations of the Ready Queue

4.13.4.1 Enqueue Operation

- When a process **transitions to the ready state** (e.g., after I/O completion or creation), its PCB is **appended to the tail** of the ready queue.
 - Steps:
 1. The **tail pointer** of the queue header is updated to point to the new PCB.
 2. The new PCB's **next** pointer is set to **NULL**.
 3. The previous last PCB's **next** pointer is updated to point to the new PCB.

4.13.4.2 Dequeue Operation

- When the **CPU scheduler selects a process** for execution, its PCB is **removed from the head** of the ready queue.
 - Steps:
 1. The **head pointer** is updated to point to the next PCB in the queue.
 2. If the queue becomes empty, both **head and tail pointers** are set to NULL.

4.13.5 Key Takeaways

1. **Definition:**
 - The ready queue is a **main memory-resident linked list** of PCBs for processes in the **ready state**.
2. **Function:**
 - Manages **processes competing for CPU time**, enabling **multiprogramming** and **efficient CPU utilization**.
3. **Degree of Multiprogramming:**
 - Directly tied to the **number of processes in the ready queue**.
 - Higher degree = more processes ready to run, but requires **balancing** to avoid overhead.
4. **Implementation:**
 - **Linked list** structure allows **dynamic insertion/removal** of PCBs.
 - **Queue header** (with head and tail pointers) ensures **O(1)** access to the first and last elements.
5. **Process Ordering:**
 - Processes are **not stored in PID order**; the scheduler determines execution sequence based on **scheduling algorithms** (e.g., Round Robin, Priority Scheduling).

4.14 Recording of Operating Systems Week 3 - Live Session on 26-03-28

4.14.1 1. Introduction to Processes

4.14.1.1 1.1 Definition of a Process

- A **process** is a program in execution.
- It is the fundamental unit of work in an operating system (OS).
- Every process has:
 - A **program counter (PC)** - Indicates the next instruction to be executed.
 - A **process state** - Defines the current status (e.g., running, ready, waiting).
 - A **process map** - Logical division of memory for efficient management.

4.14.1.2 1.2 Process Map (Memory Layout) The **process map** is a **logical** (not physical) division of memory into sections for efficient access and management.

4.14.1.2.1 1.2.1 Sections of a Process Map

Section	Description	Example
Stack	Stores pure local variables , function call references, and return addresses. Memory is automatically allocated/deallocated when entering/exiting a function.	<code>int x = 10;</code> (local variable)
Heap	Used for dynamic memory allocation at runtime.	<code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> (C), <code>new</code> (C++, Java, .NET)

Section	Description	Example
Code/Text	Contains the executable instructions (machine code) of the program.	Compiled binary of the program
Data	Stores initialized static/global variables .	<code>static int y = 5;</code>
BSS (Block Started by Symbol)	Stores uninitialized static/global variables .	<code>static int z;</code> (no initialization)

4.14.1.2.2 1.2.2 Key Differences: Data vs. BSS

- **Data Section:**
 - Stores **initialized** static/global variables.
 - Example: `int count = 10;`
- **BSS Section:**
 - Stores **uninitialized** static/global variables.
 - Example: `int count;` (no initialization)
 - **Why separate?**
 - * BSS does not occupy space in the executable file (saves disk space).
 - * The OS initializes BSS variables to **zero** at runtime.

4.14.1.2.3 1.2.3 Pure vs. Non-Pure Local Variables

- **Pure Local Variables:**
 - **Automatic variables** (default in functions).
 - Memory allocated on the **stack**.
 - Lifetime: Only within the function scope.
 - Example: `int x = 10;` (inside a function)
- **Non-Pure Local Variables:**
 - Modified with **storage class specifiers** (`static`, `register`).
 - Example:
 - * `static int y = 20;` -> Stored in **Data/BSS** (not stack).
 - * `register int z = 30;` -> Stored in a **CPU register** (if possible).

4.14.1.2.4 1.2.4 Why Do We Need a Process Map?

- **Efficient Memory Access:**
 - Without sections, the OS would **scan the entire memory** to find data (inefficient).
 - With sections, the OS **knows where to look** (e.g., stack for local variables, heap for dynamic allocations).
- **Analogy:**
 - Like organizing people in a room by designation (directors in front, programmers at the back) for quick access.

4.14.2 2. Process States and Transitions

4.14.2.1 2.1 Process State Diagram A process transitions between states based on OS scheduling:

New -> Ready -> Running -> Terminated

```

      ↑       ↓
    Ready  Waiting (I/O)

```

4.14.2.1.1 2.1.1 State Definitions

State	Description
New	Process is created but not yet admitted to the ready queue.
Ready	Process is loaded in main memory and waiting for CPU time.
Running	Process is executing on the CPU.
Waiting (Blocked)	Process is waiting for I/O or an event (e.g., file read, user input).
Terminated	Process has finished execution or was killed .

4.14.2.1.2 2.1.2 Memory Allocation in States

- **New State:**
 - Process exists in **secondary memory (disk)**.
 - No main memory allocation yet.
- **Ready State:**
 - Process is **loaded into main memory (RAM)**.
 - Waiting for CPU time (standby mode).
- **Running State:**
 - Process is **actively using CPU and main memory**.
- **Terminated State:**
 - All **memory and resources are deallocated**.

4.14.2.1.3 2.1.3 Example: ATM Queue Analogy

- **New State:** Person enters the ATM area but hasn't joined the queue.
- **Ready State:** Person is **standing in the queue** (waiting for turn).
- **Running State:** Person is **using the ATM machine**.
- **Waiting State:** Person is **waiting for cash dispensing** (I/O operation).
- **Terminated State:** Person **leaves after completing the transaction**.

4.14.3 3. Process Control Block (PCB)

4.14.3.1 3.1 Definition

- Every process is represented by a **Process Control Block (PCB)** (also called **Task Control Block** in some OS).
- PCB is a **data structure** that stores all information about a process.

4.14.3.2 3.2 Contents of a PCB

Field	Description
Process ID (PID)	Unique identifier for the process.
Process State	Current state (New, Ready, Running, etc.).
Program Counter (PC)	Address of the next instruction to execute.
CPU Registers	Values of registers when the process was last running.
Memory Limits	Information about memory allocation (stack, heap, etc.).
I/O Status	List of I/O devices allocated to the process.
Scheduling Information	Priority, time quantum, etc.
Parent Process ID (PPID)	PID of the parent process.

4.14.3.3 3.3 Role of PCB in Context Switching

- When the **scheduler** switches from one process to another (**context switching**), it:
 1. **Saves** the current process's state (PC, registers, etc.) into its **PCB**.
 2. **Loads** the next process's state from its **PCB** into the CPU.
 3. **Resumes execution** of the new process.

4.14.3.3.1 3.3.1 Example: ATM Context Switch

- **Process P1** is using the ATM (Running).
- A **time limit** is reached (or an **interrupt** occurs).
- The OS **saves P1's state** (e.g., remaining transaction steps) in its PCB.
- **Process P2** is loaded from its PCB and starts using the ATM.

4.14.4 4. Reasons for Context Switching

The OS performs context switching for:

1. **Multitasking**: - Allow multiple processes to **share CPU time**. - Prevents one process from **monopolizing the CPU**.
2. **Interrupt Handling**: - Hardware/software interrupts (e.g., I/O completion, timer expiry).
3. **User/Kernel Mode Switching**: - When a process makes a **system call** (e.g., file access), it switches from **user mode** -> **kernel mode**. - Only the **kernel** can perform privileged operations (e.g., process creation, memory allocation).

4.14.5 5. The init Process (Parent of All Processes)

4.14.5.1 5.1 Definition

- **init** (PID = 1) is the **first process** created by the kernel during boot.
- It is the **ancestor of all processes** in Unix/Linux systems.

4.14.5.2 5.2 Roles of init

- **Initializes the system** by executing startup scripts.
- **Sets up environment variables**.
- **Manages system services (daemons)**.
- **Handles run levels** (different operational modes).

4.14.5.3 5.3 Run Levels in Linux

Run Level	Description
0	Halt - Shuts down the system.
1	Single-user mode - Only root can log in (maintenance mode).
2	Multi-user mode (no networking) - Multiple users, but no network services.
3	Multi-user mode (with networking) - Full operational state (default for servers).
4	Unused (Customizable).
5	Graphical mode (X11) - GUI login (default for desktops).
6	Reboot - Restarts the system.

4.14.5.4 5.4 Daemons (Background Processes)

- **Definition:** A **daemon** is a background process that runs **independently of user sessions**.
- **Characteristics:**
 - Typically **start at boot** and run until shutdown.
 - Often have names ending with **d** (e.g., **sshd**, **cron**, **syslogd**).
 - Perform **system tasks** (e.g., logging, scheduling, network services).
- **Examples:**
 - **sshd** - Secure Shell daemon (handles remote logins).
 - **cron** - Schedules periodic tasks.
 - **syslogd** - Manages system logs.

4.14.6 6. Process Creation

4.14.6.1 6.1 Process Hierarchy

- Every process (except **init**) has a **parent process**.
- The **parent process** creates **child processes** using system calls (e.g., **fork()** in Unix).

4.14.6.2 6.2 Steps in Process Creation

1. The OS assigns a **unique Process ID (PID)**.
2. A new **entry is added to the process table**.
3. **Memory is allocated** (stack, heap, data segments).
4. **PCB is initialized** (PC, registers, state, etc.).
5. The process enters the **Ready Queue**.

4.14.6.3 6.3 Example: **fork()** System Call (Unix/Linux)

- **fork()** creates a **child process** that is an **exact copy** of the parent.
- **Return Values:**
 - **Parent Process:** Returns the **child's PID**.
 - **Child Process:** Returns **0**.
 - **Error:** Returns **-1** (child not created).

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork(); // Create a child process

    if (pid < 0) {
        printf("Fork failed.\n");
    } else if (pid == 0) {
        // Child process
        printf("Child Process: PID = %d, Parent PID = %d\n", getpid(), getppid());
    } else {
        // Parent process
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    }
}
```

```
return 0;
}
```

4.14.6.3.1 6.3.1 Code Example (C)

4.14.6.3.2 6.3.2 Output Explanation

- **Parent Process:**
 - Prints its own PID and the child's PID.
- **Child Process:**
 - Prints its PID (new) and the parent's PID (same as parent's PID).

4.14.7 7. Process Termination

4.14.7.1 7.1 Normal Termination

- Occurs when:
 - The process **completes its task** (e.g., `exit()` system call).
 - The **parent process requests termination** (e.g., `kill()`).
- **Steps:**
 1. OS **releases all resources** (memory, open files, I/O devices).
 2. Process is **removed from the process table**.
 3. **Exit status** is returned to the parent (via `wait()`).

4.14.7.2 7.2 Abnormal Termination Occurs due to: 1. **Memory Unavailability:** - Process requests more memory than available -> **killed by OS**. 2. **Exceeding Time Limit:** - Process runs longer than allocated **time quantum** -> **preempted**. 3. **Protection Errors:** - Attempting to access **restricted memory/files** -> **segmentation fault**. 4. **Arithmetic Errors:** - **Division by zero** -> **abnormal termination**. 5. **I/O Failures:** - Unable to read/write to a device -> **terminated**. 6. **Parent Process Termination:** - If the **parent dies**, all its **child processes are terminated** (cascading termination). 7. **Invalid Instructions:** - Executing **illegal machine code** -> **killed by OS**. 8. **Privileged Instruction Use:** - User process tries to execute a **kernel-only instruction** -> **terminated**.

4.14.8 8. Inter-Process Communication (IPC)

4.14.8.1 8.1 Need for IPC

- **Independent Processes:**
 - Do **not communicate** with other processes.
 - Example: A text editor and a calculator running separately.
- **Cooperating Processes:**
 - **Share data** and **synchronize** actions.
 - Example: A **banking system** where:
 - * **Deposit module** updates the balance.
 - * **Withdrawal module** reads the balance.

4.14.8.2 8.2 Advantages of Cooperating Processes

1. **Modularity:** Break complex tasks into smaller processes.
2. **Information Sharing:** Multiple processes access shared data (e.g., database).
3. **Computation Speedup:** Parallel execution (e.g., multi-threaded applications).
4. **Convenience:** Easier for users to perform multiple tasks simultaneously.

4.14.8.3 8.3 IPC Mechanisms

Mechanism	Description	Example
Shared Memory	Processes read/write to a common memory region .	Google Drive (multiple users access the same file).
Message Passing Pipes	Processes exchange messages via OS. Unidirectional communication between processes.	WhatsApp (sender -> receiver). <code>ps wc -l</code> (output of <code>ps</code> is input to <code>wc</code>).
Sockets	Network-based communication (local or remote).	Web browsers communicating with servers.
Files	Processes read/write to a common file .	Log files shared between processes.

4.14.8.3.1 8.3.1 Example: Pipes in Linux

- **Command:** `ps -aux | grep "python" | wc -l`
 - `ps -aux` -> Lists all processes.
 - `grep "python"` -> Filters processes containing “python”.
 - `wc -l` -> Counts the number of lines (processes).
- **How it works:**
 - Output of `ps` -> Input to `grep`.
 - Output of `grep` -> Input to `wc`.

4.14.9 9. Process Scheduling Queues

4.14.9.1 9.1 Types of Queues

Queue	Description	Location
Job Queue	All processes in the system.	Secondary memory (disk).
Ready Queue	Processes ready to execute (waiting for CPU).	Main memory (RAM).
Device Queue	Processes waiting for I/O (e.g., disk, printer).	Main memory (RAM).

4.14.9.2 9.2 Process Movement Between Queues

1. **New -> Ready Queue:**
 - Process is **loaded into main memory**.
2. **Ready Queue -> Running:**
 - **Short-term scheduler** selects a process for CPU.
3. **Running -> Waiting (I/O Queue):**
 - Process requests I/O -> **moved to device queue**.
4. **Waiting -> Ready Queue:**
 - I/O completes -> process **returns to ready queue**.
5. **Running -> Terminated:**
 - Process **finishes execution** or is **killed**.

4.14.10 10. Process Schedulers

4.14.10.1 10.1 Types of Schedulers

Scheduler	Role	Speed	Frequency
Long-Term (Job Scheduler)	Selects processes from secondary memory to load into ready queue .	Slow	Infrequent (minutes/hours).
Short-Term (CPU Scheduler)	Selects a process from ready queue to run on CPU .	Very Fast	Frequent (milliseconds).
Medium-Term (Swapping Scheduler)	Moves processes between main memory and disk to manage load.	Medium	Occasional (seconds).

4.14.10.1.1 10.1.1 Long-Term Scheduler (Job Scheduler)

- **Responsibilities:**
 - Controls the **degree of multiprogramming** (number of processes in memory).
 - Decides which processes are **admitted to the ready queue**.
- **Example:**
 - If too many **I/O-bound processes** are in memory, it may **delay CPU-bound processes**.

4.14.10.1.2 10.1.2 Short-Term Scheduler (CPU Scheduler)

- **Responsibilities:**
 - **Selects the next process to run** on the CPU.
 - Must be **fast** to avoid CPU idle time.
- **Example:**
 - Like a **security guard** at an ATM deciding who goes next.

4.14.10.1.3 10.1.3 Medium-Term Scheduler (Swapping Scheduler)

- **Responsibilities:**
 - **Swaps out** processes that are **idle for too long** (to free memory).
 - **Swaps in** processes when memory is available.
- **Example:**
 - If a process is **waiting for user input** for too long, it may be **moved to disk**.

4.14.10.2 10.2 Comparison of Schedulers

Feature	Long-Term	Short-Term	Medium-Term
Speed	Slow	Very Fast	Medium
Frequency	Low	High	Moderate
Primary Function	Load processes into memory	Allocate CPU to processes	Manage memory load

Feature	Long-Term	Short-Term	Medium-Term
Impact on Performance	Affects multiprogramming degree	Affects CPU utilization	Affects memory availability

4.14.11 11. Summary of Key Concepts

1. **Process Map:** Logical memory division (stack, heap, code, data, BSS).
2. **Process States:** New -> Ready -> Running -> Waiting -> Terminated.
3. **PCB:** Stores process metadata for context switching.
4. **init Process:** Parent of all processes (PID = 1).
5. **Process Creation:** `fork()` creates child processes.
6. **Process Termination:** Normal (completion) or abnormal (errors).
7. **IPC:** Shared memory, message passing, pipes, sockets.
8. **Scheduling Queues:** Job, Ready, Device queues.
9. **Schedulers:** Long-term (job), short-term (CPU), medium-term (swapping).

4.15 Schedulers

4.15.1 1. Introduction to Schedulers

- **Definition:** A **scheduler** is a system software component responsible for selecting processes from a **scheduling queue** (e.g., job queue, ready queue, device queue).
- **Purpose:** Determines which process(es) should be allocated system resources (e.g., CPU, memory) at a given time.

4.15.2 2. Types of Schedulers

There are **three primary types of schedulers** in an operating system: 1. **Long-Term Scheduler (Job Scheduler)** 2. **Short-Term Scheduler (CPU Scheduler)** 3. **Medium-Term Scheduler**

Each scheduler operates at different levels of process management and serves distinct functions.

4.15.3 3. Long-Term Scheduler (Job Scheduler)

4.15.3.1 3.1 Definition and Role

- Also known as the **job scheduler**.
- **Primary Function:** Selects processes (jobs) from the **job pool (job queue)** and loads them into **main memory (ready queue)**.
- Operates in **batch systems**, where users submit jobs that are stored in a job queue before execution.

4.15.3.2 3.2 Key Characteristics

- **Invocation Frequency:**
 - Invoked **infrequently** (only when a process terminates).
 - Termination of a process frees up memory, allowing the long-term scheduler to load a new job.
- **Decision Time:**
 - Can take **relatively longer** to decide which job to load next (compared to short-term schedulers).
 - Not required to be extremely fast since it is not called frequently.
- **Degree of Multiprogramming Control:**

- **Degree of Multiprogramming:** The number of processes residing in main memory at any given time.
- The long-term scheduler **directly controls** this degree by deciding how many jobs to admit into memory.

4.15.3.3 3.3 Example Scenario

- Suppose a batch system has **10 jobs** in the job queue but only **4 can fit in memory**.
- The long-term scheduler selects **4 jobs** to load into the **ready queue**.
- The remaining **6 jobs** stay in the job queue until memory becomes available.

4.15.4 4. Short-Term Scheduler (CPU Scheduler)

4.15.4.1 4.1 Definition and Role

- Also known as the **CPU scheduler**.
- **Primary Function:** Selects a process from the **ready queue** and allocates the **CPU** to it for execution.
- Operates in **multitasking** and **time-sharing systems**.

4.15.4.2 4.2 Key Characteristics

- **Invocation Frequency:**
 - Invoked **very frequently** (microseconds to milliseconds).
 - Triggered by **process state transitions**, such as:
 - * Running -> Waiting (e.g., I/O request)
 - * Running -> Ready (e.g., time slice expires)
 - * Waiting -> Ready (e.g., I/O completion)
- **Decision Time:**
 - Must be **extremely fast** to avoid CPU idle time.
 - Uses **scheduling algorithms** (e.g., Round Robin, Priority Scheduling) to make quick selections.
- **Presence in Systems:**
 - **Always present** in multitasking/time-sharing systems.
 - **Absent in pure batch systems** (where long-term scheduling dominates).

4.15.4.3 4.3 Comparison with Long-Term Scheduler

Feature	Long-Term Scheduler	Short-Term Scheduler
Invocation Frequency	Infrequent (process termination)	Very frequent (state transitions)
Decision Speed	Can take longer	Must be very fast
Primary Queue	Job Queue	Ready Queue
System Type	Batch systems	Multitasking/Time-sharing

4.15.5 5. Medium-Term Scheduler

4.15.5.1 5.1 Definition and Role

- Introduces an **intermediate level of scheduling** between long-term and short-term schedulers.
- **Primary Functions:**
 1. **Reduce the degree of multiprogramming** (if too many processes are in memory).
 2. **Improve process mix** (balance between I/O-bound and CPU-bound processes).

4.15.5.2 5.2 Key Mechanisms

- **Swapping:**
 - **Swap Out:** Removes a process from the **ready queue** and stores it in **secondary storage (swap space)**.
 - * The process is **temporarily suspended** (not terminated).
 - * Its state is saved to allow **resumption later**.
 - **Swap In:** Retrieves a swapped-out process from secondary storage and moves it back to the **ready queue**.
 - * The process **resumes execution from where it left off** (not from scratch).

4.15.5.3 5.3 Use Cases

1. **Reducing Degree of Multiprogramming:**
 - If the system is **overloaded** (too many processes in memory), the medium-term scheduler can **swap out** some processes to free up memory.
 - Example: Reducing from **10 processes** in memory to **8** by swapping out **2**.
2. **Balancing Process Mix:**
 - If too many **I/O-bound processes** are present, the CPU may remain idle while waiting for I/O.
 - If too many **CPU-bound processes** are present, I/O devices may remain idle.
 - The medium-term scheduler can **swap out** some processes and **swap in** others to achieve a **balanced mix**.

4.15.5.4 5.4 Swap Space

- A designated area in **secondary storage** (e.g., disk) where swapped-out processes are temporarily stored.
- Allows the system to **manage memory efficiently** without terminating processes.

4.15.6 6. Summary of Scheduler Functions

Scheduler Type	Alternative Source Name	Queue	Destination Queue	Primary Function	Invocation Frequency	Decision Speed	Typical System
Long-Term	Job Scheduler	Job Queue	Ready Queue	Selects jobs to load into memory	Infrequent	Moderate	Batch systems
Short-Term	CPU Scheduler	Ready Queue	CPU (Execution)	Selects next process to run on CPU	Very Frequent	Very Fast	Multitasking/Time-sharing
Medium-Term	-	Ready Queue	Swap Space	Swaps processes in/out to manage memory load	Moderate	Moderate	General-purpose systems

4.15.7 7. Key Takeaways

1. **Long-Term Scheduler:**
 - Controls **degree of multiprogramming**.
 - Operates in **batch systems**.
 - Invoked **infrequently** (on process termination).
2. **Short-Term Scheduler:**
 - **Allocates CPU** to processes from the ready queue.
 - Must be **very fast** due to frequent invocations.

- Present in **multitasking/time-sharing systems**.
3. **Medium-Term Scheduler:**
- **Swaps processes** between memory and secondary storage.
 - Helps **balance system load** and **optimize resource usage**.
 - Introduces **intermediate scheduling** for better process management.

4.16 Shared Memory

4.16.1 Introduction to Shared Memory

- **Definition:** Shared memory is an **inter-process communication (IPC)** mechanism where multiple processes communicate by accessing a **common memory region**.
- **Core Idea:**
 - Processes establish a **shared memory region** to exchange data.
 - One process **creates** the shared memory segment, while others **attach** it to their address spaces.
 - The operating system (OS) **relaxes access restrictions** to allow controlled access to this shared region.

4.16.2 Mechanism of Shared Memory

4.16.2.1 1. Creation and Attachment of Shared Memory

- **Process Roles:**
 - **Creator Process (e.g., P1):**
 - * Allocates a **shared memory segment** in its address space.
 - * Other processes (e.g., P2, P3, P4) **attach** this segment to their own address spaces.
 - **Participating Processes:**
 - * Use **predefined API function calls** to attach to the shared memory segment.
 - * Once attached, they can **read/write** to the shared region as if it were part of their local memory.
- **Key Observations:**
 - Normally, the OS **prevents processes from accessing each other's address spaces** for security.
 - Shared memory **selectively allows access** only to the designated shared region.
 - **No other segments** of a process's address space become accessible to others.

4.16.2.2 2. Data Exchange via Shared Memory

- **Communication Process:**
 1. A process (e.g., P1) **writes data** into the shared memory segment.
 2. Other processes (e.g., P2, P3) **read** the data from the same segment.
 3. The shared region acts like a **shared buffer** for inter-process communication.
- **Example Scenario:**
 - **P1** creates a shared memory segment.
 - **P2, P3, P4** attach to it.
 - If **P1** writes "Hello" to the shared segment, **P2, P3, P4** can read "Hello" from their attached regions.

4.16.2.3 3. Termination of Shared Memory

- **Cleanup:**
 - After communication completes, the **creator process (P1)** must **delete** the shared memory segment to free system resources.
 - Failure to do so may lead to **memory leaks**.

4.16.3 Synchronization Requirements

- **Problem:** Without synchronization, **race conditions** can occur:
 - Multiple processes writing **simultaneously** may **overwrite** each other's data.
 - A process reading while another is writing may get **inconsistent/partial data**.
- **Solution:**
 - Use **synchronization mechanisms** (e.g., **semaphores, mutexes**) to ensure:
 - * Only **one process writes at a time**.
 - * **Readers do not access** the segment while a writer is active.

4.16.4 Advantages of Shared Memory

4.16.4.1 1. High-Speed Communication

- **Minimal Overhead:**
 - System calls (via API functions) are **only needed during setup** (creation/attachment).
 - Once established, **read/write operations** are as fast as **normal memory accesses** (no OS intervention per operation).
- **Comparison with Other IPC Mechanisms:**
 - **Message passing** (e.g., pipes, sockets) requires **kernel involvement** for every send/receive, slowing communication.
 - Shared memory **bypasses the kernel** after setup, making it **faster for bulk data transfer**.

4.16.4.2 2. Efficient Bulk Data Transfer

- **Use Case:** Ideal for scenarios where processes need to exchange **large datasets** (e.g.,:
 - **Multimedia processing** (video frames, audio buffers).
 - **Scientific computing** (shared matrices, simulation data).
 - **Databases** (shared caches, transaction logs).
- **Why?** Avoids the **serialization/deserialization** overhead of message passing.

4.16.5 Disadvantages of Shared Memory

4.16.5.1 1. Need for Explicit Synchronization

- **Challenge:**
 - Developers must **manually implement synchronization** to prevent:
 - * **Data corruption** (simultaneous writes).
 - * **Inconsistent reads** (reading while writing).
- **Complexity:**
 - Requires careful design to avoid **deadlocks, starvation, or priority inversion**.

4.16.5.2 2. Unsuitability for Distributed Systems

- **Limitation:**
 - Shared memory **relies on a common physical memory space**, which is **not available in distributed systems** (e.g., cluster computing, cloud applications).
- **Alternatives for Distributed Systems:**
 - **Message passing** (e.g., MPI, sockets).
 - **Distributed shared memory (DSM)** emulation (complex and often slower).

4.16.5.3 3. Potential for Memory Leaks

- **Risk:**
 - If the creator process **crashes or forgets to delete** the shared segment, it remains in the system, wasting memory.
- **Mitigation:**
 - Use **reference counting** or **OS-level cleanup** (e.g., Linux `shmctl` with `IPC_RMID`).

4.16.6 Summary of Key Concepts

Aspect	Details
Definition	IPC mechanism where processes share a common memory region.
Creation	One process allocates the segment; others attach it.
Access	Processes read/write to the segment like local memory.
Synchronization	Required to prevent race conditions (e.g., semaphores).
Advantages	Fast, efficient for bulk data, minimal kernel overhead after setup.
Disadvantages	Needs synchronization, not distributed-system-friendly, risk of memory leaks.
Cleanup	Creator must delete the segment post-use.

4.16.7 Practical Example

4.16.7.1 Scenario: Producer-Consumer Problem

1. **Producer Process (P1):**
 - Creates a shared memory segment.
 - Writes data (e.g., sensor readings) to the segment.
2. **Consumer Processes (P2, P3):**
 - Attach to the segment.
 - Read data from the segment (e.g., for analysis).
3. **Synchronization:**
 - A **mutex** ensures only one producer/consumer accesses the segment at a time.
 - **Condition variables** signal when new data is available.

4.16.8 API Functions (Conceptual Overview)

While exact functions vary by OS, typical operations include: 1. **Creation:** - `shmget()` (System V) or `shm_open()` (POSIX) - Allocates a shared segment. 2. **Attachment:** - `shmat()` (System V) or `mmap()` (POSIX) - Maps the segment into a process's address space. 3. **Detachment:** - `shmdt()` - Removes the segment from a process's address space. 4. **Deletion:** - `shmctl()` (with `IPC_RMID`) - Marks the segment for destruction.

4.16.9 When to Use Shared Memory?

Use Case	Suitable?	Reason
High-frequency data exchange	Yes	Low latency after setup.
Large dataset sharing	Yes	Avoids serialization overhead.
Distributed applications	No	No shared physical memory.
Simple IPC between local processes	Yes	Efficient for single-machine multi-processing.

4.16.10 Conclusion

Shared memory is a **powerful IPC mechanism** for **local, high-performance communication** but requires **careful synchronization** and is **limited to non-distributed systems**. It excels in scenarios demanding **low-latency, bulk data transfer** but introduces complexity in managing concurrent access. Proper cleanup is essential to avoid resource leaks.

4.17 States of a Process

4.17.1 Introduction to Process States

- A **process** undergoes multiple **states** during its **life cycle** in an operating system.
- Each state represents a distinct phase in the process's execution journey.
- **State transitions** occur due to specific triggers (e.g., system events, resource availability, or scheduling decisions).

4.17.2 Process States in Detail

4.17.2.1 1. New State

- **Definition:** The initial state of a process when it is **created** but **not yet loaded into main memory**.
- **Key Characteristics:**
 - The process exists in **secondary memory** (e.g., disk).
 - It has **not been admitted** into the system's main memory for execution.
 - The process is **not eligible** for CPU allocation yet.
- **Transition Trigger:**
 - The process moves to the **ready state** when it is **loaded into main memory** by the OS.

4.17.2.2 2. Ready State

- **Definition:** The process is **loaded into main memory** and **ready for execution** but **not currently running**.
- **Key Characteristics:**
 - The process resides in a **ready queue** (a data structure in main memory).
 - The **ready queue** holds all processes awaiting CPU allocation.
 - The process is **fully prepared** to execute but must **wait for the CPU** to be assigned.
- **Transition Triggers:**
 - Moves to the **running state** when the **CPU is allocated** to it (via the **scheduler**).
 - May return to the **ready state** if **preempted** (e.g., due to time quantum expiration or higher-priority process).

4.17.2.3 3. Running State

- **Definition:** The process is **actively executing** on the CPU.
- **Key Characteristics:**
 - The CPU executes the process's instructions **one by one**.
 - In a **multiprocessor system**, multiple processes can be in the running state simultaneously (one per CPU core).
- **Transition Triggers:**
 1. **To Waiting State:**
 - If the process **requests an I/O operation** (e.g., reading from disk, writing to a file).
 - If the process **waits for an event** (e.g., a signal, synchronization condition).
 - The process is **removed from the ready queue** and placed in a **device-specific queue** (e.g., I/O queue).

2. To Ready State:

- If the process is **preempted** (e.g., due to a **time slice expiration** in round-robin scheduling).
- If an **interrupt** occurs (e.g., a **system call** or **hardware interrupt**).

3. To Terminated State:

- If the process **completes execution** (all instructions are finished).
- The OS **reclaims all resources** (memory, file handles, etc.) allocated to the process.

4.17.2.4 4. Waiting State (Blocked State)

- **Definition:** The process is **waiting for an event** (e.g., I/O completion, signal, resource availability).
- **Key Characteristics:**
 - The process **cannot execute** until the event occurs.
 - It is **not in the ready queue** but in a **device-specific queue** (e.g., I/O queue).
 - The process **does not consume CPU time** while waiting.
- **Transition Triggers:**
 - Moves back to the **ready state only after the event completes** (e.g., I/O operation finishes).
 - **Never transitions directly to the running state**-must go through the ready state first.

4.17.2.5 5. Terminated State

- **Definition:** The process has **finished execution** and is **no longer active**.
- **Key Characteristics:**
 - All instructions have been executed.
 - The process is **removed from the system**.
 - The OS **deallocates all resources** (memory, CPU, files, etc.) associated with the process.
- **Transition Trigger:**
 - Occurs when the process **explicitly terminates** (e.g., `exit()` system call) or **completes its task**.

4.17.3 Process State Transitions

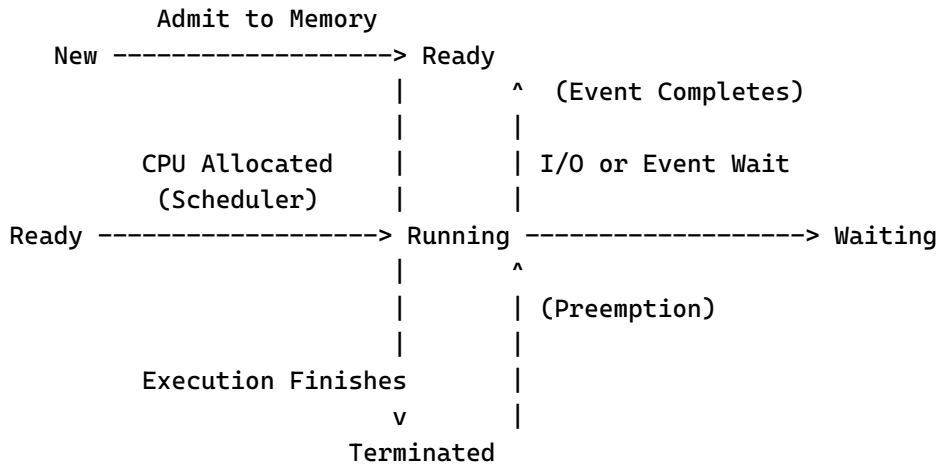
4.17.3.1 Valid State Transitions The following transitions are **permitted** in a process's life cycle:

1. **New -> Ready**
 - Trigger: Process is **loaded into main memory**.
2. **Ready -> Running**
 - Trigger: Process is **selected by the CPU scheduler**.
3. **Running -> Ready**
 - Trigger: Process is **preempted** (e.g., time quantum expires, higher-priority process arrives).
4. **Running -> Waiting**
 - Trigger: Process **requests I/O or waits for an event**.
5. **Waiting -> Ready**
 - Trigger: The **waited-for event completes** (e.g., I/O operation finishes).
6. **Running -> Terminated**
 - Trigger: Process **finishes execution** or is **forcefully terminated**.

4.17.3.2 Invalid State Transitions The following transitions **never occur**:

1. **Waiting -> Running**
 - A process **must** go through the **ready state** before running again.
2. **Ready -> Waiting**
 - A process can **only enter the waiting state from the running state** (not directly from ready).
 - Reason: A process can only **initiate a wait** while it is **executing** (i.e., in the running state).

4.17.4 State Transition Diagram (Conceptual)



4.17.5 Key Observations

1. Preemption:

- A running process can be **forced back to ready** (preempted) due to:
 - **Time quantum expiration** (in time-sharing systems).
 - **Higher-priority process** becoming ready.
 - **Interrupts** (e.g., hardware signals).

2. I/O Bound vs. CPU Bound Processes:

- **I/O-bound processes** spend more time in the **waiting state**.
- **CPU-bound processes** spend more time in the **running/ready states**.

3. Resource Management:

- The OS **reclaims resources** only in the **terminated state**.
- A process in the **waiting state** does **not hold the CPU** but may hold other resources (e.g., memory, files).

4.17.6 Examples of State Transitions

1. Process Creation & Execution:

- A user runs a program -> **New** (created) -> **Ready** (loaded into memory) -> **Running** (CPU allocated).

2. I/O Operation:

- A running process requests disk read -> **Running** -> **Waiting** (until I/O completes) -> **Waiting** -> **Ready** (I/O done) -> **Ready** -> **Running** (when CPU is free).

3. Preemption Due to Time Slice:

- A process runs for its time quantum -> **Running** -> **Ready** (preempted) -> **Ready** -> **Running** (when rescheduled).

4. Process Termination:

- A process finishes its task -> **Running** -> **Terminated** (resources freed).

4.18 Types of Processes

4.18.1 Introduction

- This lecture focuses on **categorizing processes** based on **resource utilization** and analyzing **system performance** in the context of different process types.
- The categorization is primarily based on how processes utilize **CPU and I/O resources**.

- The goal is to understand how the system performs when different types of processes execute **individually or simultaneously**.

4.18.2 Categorization of Processes by Resource Utilization

Processes can be classified into **two primary types** based on their resource usage patterns:

4.18.2.1 1. CPU-Bound Processes

4.18.2.1.1 Definition:

- A **CPU-bound process** is one that **spends the majority of its execution time performing computations**.
- It **heavily utilizes the processor (CPU)** for most of its lifecycle.

4.18.2.1.2 Characteristics:

- **High CPU Usage:** The process predominantly engages the CPU for computational tasks (e.g., mathematical calculations, data processing, algorithm execution).
- **Low I/O Activity:** Generates **very few input/output (I/O) requests**.
 - Minimal interaction with I/O devices (e.g., disk drives, network interfaces, keyboards).
 - Spends **negligible time waiting for I/O operations** to complete.
- **Processor-Centric:** The performance of the process is **limited by CPU speed and availability** rather than I/O operations.

4.18.2.1.3 Impact on System Performance:

- **Keeps CPUs Busy:** Since CPU-bound processes continuously require CPU cycles, they **maximize processor utilization**.
- **Underutilizes I/O Devices:** Due to minimal I/O requests, **I/O devices (e.g., disks, printers) remain idle**.
- **Potential Bottleneck:** If too many CPU-bound processes run simultaneously, they may **compete for CPU time**, leading to **longer wait times in the ready queue**.

4.18.2.1.4 Examples:

- Scientific computations (e.g., weather modeling, physics simulations).
- Data encryption/decryption algorithms.
- Compilation of large programs.
- Matrix multiplications or other intensive mathematical operations.

4.18.2.2 2. I/O-Bound Processes

4.18.2.2.1 Definition:

- An **I/O-bound process** is one that **spends the majority of its execution time performing input/output operations**.
- It **frequently interacts with I/O devices** and spends **minimal time on computations**.

4.18.2.2.2 Characteristics:

- **High I/O Activity:** Generates a **large number of I/O requests** (e.g., reading/writing files, network communication, user input).

- **Low CPU Usage:** Spends **very little time performing computations**; most of its time is spent **waiting for I/O operations to complete**.
- **I/O-Centric:** The performance of the process is **limited by the speed of I/O devices** (e.g., disk latency, network bandwidth) rather than CPU speed.

4.18.2.2.3 Impact on System Performance:

- **Keeps I/O Devices Busy:** Continuously generates I/O requests, **maximizing utilization of I/O devices**.
- **Underutilizes CPUs:** Since the process spends most of its time waiting for I/O, **CPUs remain idle** for extended periods.
- **Potential Bottleneck:** If too many I/O-bound processes run simultaneously, they may **overwhelm I/O devices**, leading to **queuing delays** for I/O operations.

4.18.2.2.4 Examples:

- Text editors (frequent user input/output).
- Web servers (handling multiple client requests, reading/writing files).
- Database applications (frequent disk reads/writes).
- Multimedia applications (streaming audio/video, requiring constant I/O).

4.18.3 Resource Utilization and System Balance

4.18.3.1 Implications of Process Type Dominance The **distribution of CPU-bound and I/O-bound processes** in a system directly impacts **overall resource utilization** and **system efficiency**:

Scenario	CPU Utilization	I/O Device Utilization	System Imbalance
Mostly CPU-Bound	High (CPUs are busy)	Low (I/O devices idle)	I/O resources wasted; CPUs may become bottleneck.
Mostly I/O-Bound	Low (CPUs are idle)	High (I/O devices busy)	CPU cycles wasted; I/O devices may become bottleneck.

4.18.3.2 Optimal Process Mix

- A **balanced mix** of CPU-bound and I/O-bound processes is **essential for efficient system performance**.
 - **CPU-bound processes** keep the **processors occupied**.
 - **I/O-bound processes** keep the **I/O devices occupied**.
- **Goal:** Ensure **neither CPUs nor I/O devices are underutilized** or idle for prolonged periods.

4.18.3.3 Benefits of a Balanced Mix

- **Maximized Throughput:** Both CPUs and I/O devices are **actively utilized**, reducing idle time.
- **Reduced Bottlenecks:** Prevents **CPU starvation** (too many I/O-bound processes) or **I/O congestion** (too many CPU-bound processes).
- **Improved Responsiveness:** Systems with a good mix can **handle diverse workloads** (e.g., interactive applications alongside batch processing).

4.18.4 Practical Considerations

4.18.4.1 Scheduling Challenges

- Operating systems must **schedule processes** to maintain balance:
 - **CPU-bound processes** should not **monopolize the CPU** (e.g., using **time-sharing** or **priority scheduling**).
 - **I/O-bound processes** should not **overload I/O devices** (e.g., using **I/O scheduling algorithms** like **SCAN** or **SSTF** for disk access).
- **Multiprogramming**: Running a mix of process types **improves resource utilization** by overlapping CPU and I/O operations.

4.18.4.2 Real-World Applications

- **Servers**: Often run a mix of **I/O-bound** (handling client requests) and **CPU-bound** (processing data) tasks.
- **Desktop Systems**: Typically include **interactive I/O-bound** processes (e.g., GUI applications) and **CPU-bound** background tasks (e.g., virus scans).
- **High-Performance Computing (HPC)**: Dominated by **CPU-bound** processes but may include **I/O-bound** tasks for data storage/retrieval.

4.18.5 Summary of Key Concepts

1. **CPU-Bound Processes**:
 - Dominated by **computation**; minimal I/O.
 - **Maximizes CPU usage**; underutilizes I/O.
 - Examples: Scientific computing, encryption.
2. **I/O-Bound Processes**:
 - Dominated by **I/O operations**; minimal CPU usage.
 - **Maximizes I/O usage**; underutilizes CPU.
 - Examples: Web servers, text editors.
3. **System Performance**:
 - **Imbalance** (too many of one type) leads to **resource waste**.
 - **Balanced mix** ensures **efficient utilization** of both CPUs and I/O devices.
4. **Scheduling Strategies**:
 - OS must **balance process types** to avoid bottlenecks.
 - **Multiprogramming** helps overlap CPU and I/O operations.

4.19 What is a Process

4.19.1 Introduction to Processes

- **Context**: In any computer system, hundreds of programs run simultaneously.
- **Role of the Operating System (OS)**:
 - Responsible for executing these programs.
 - Programs can be:
 - * **User programs**: Initiated by users.
 - * **System programs**: Run on behalf of the OS itself.

4.19.1.1 Terminology

- **Batch Operating Systems**: Programs are referred to as **jobs**.
- **Time-shared/Multitasking Systems**: Programs are called **user programs** or **tasks**.
- **Synonyms in Literature**:
 - The terms **jobs**, **processes**, and **tasks** are often used interchangeably.
 - All terms refer to the same fundamental concept.

4.19.2 Definition of a Process

- **Core Definition:** A process is a **program in execution**.
 - A program loaded into **main memory** and executed by the **processor**.
 - Execution progresses **sequentially**:
 - * Each instruction is fetched by the CPU and executed.
 - * May involve **data manipulation** and **memory read/write operations**.

4.19.2.1 Passive vs. Active Entities

- **Program:**
 - **Passive entity:** A file stored on **secondary storage** (e.g., hard disk).
 - Contains instructions but **cannot execute independently**.
- **Process:**
 - **Active entity:** Executes in the computing environment.
 - **Transformation from Program to Process:**
 - * Occurs when the **executable file** is loaded into **main memory**.
 - * The processor then executes the instructions line by line.

4.19.2.2 Initiating a Process

- **Graphical User Interface (GUI):**
 - Double-clicking an application's icon loads and executes the program.
- **Command Line Interface (CLI):**
 - Executed via commands (e.g., `./a.out` in Linux).

4.19.3 Multiple Processes from a Single Program

- **Key Idea:** A single program can spawn **multiple independent processes**.
- **Example:**
 1. A C program (`sample.c`) is written and compiled into an executable (`a.out`).
 2. The same executable (`./a.out`) is run in **four separate terminals**.
 3. **Result:**
 - Each execution creates a **distinct process**.
 - The OS recognizes each instance of `a.out` as a **separate, independent process**.
 - All processes perform the same task but are treated as unique entities.

4.19.4 Components of a Process

A process consists of several key sections, each serving a specific purpose:

4.19.4.1 1. Text Section (Program Section)

- Contains the **executable code** (machine instructions) of the program.
- Also referred to as the **code segment**.
- **Read-only** in most systems to prevent accidental modifications.

4.19.4.2 2. Current Activity (Process State)

- Defined by:
 - **Program Counter (PC):**
 - * Holds the **address of the next instruction** to be executed.

- **Processor Registers:**
 - * Store temporary data and intermediate results.
 - * Examples:
 - **Accumulator:** Used for arithmetic/logic operations.
 - **Index Registers:** Used for addressing memory.
 - **General-Purpose Registers:** Vary by **computer architecture**.
- **Significance:** The contents of the PC and registers define the **current context** of the process.

4.19.4.3 3. Stack Section

- Stores **temporary data** required for function execution:
 - **Function Parameters:** Arguments passed to functions.
 - **Return Addresses:** Memory addresses to return to after a function call.
 - **Local Variables:** Variables declared within a function's scope.
- **Behavior:**
 - **Grows downward** in memory (from higher to lower addresses).
 - **Dynamic:** Expands and contracts as functions are called and return.

4.19.4.4 4. Data Section

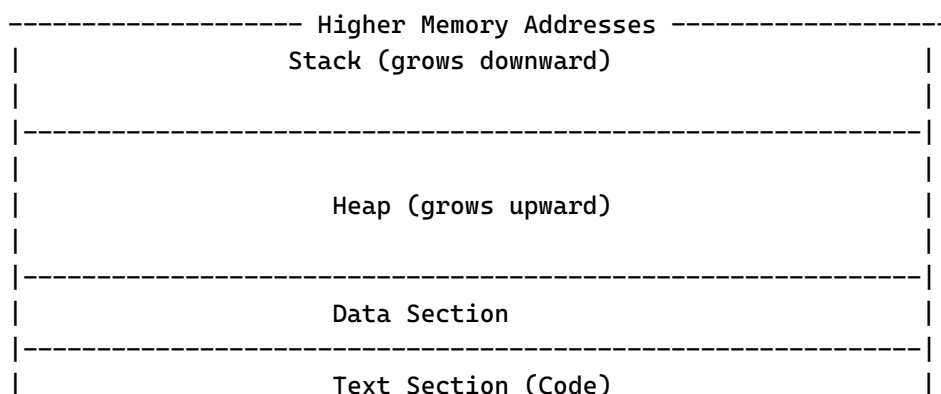
- Contains **global variables**:
 - Variables declared outside any function (global scope).
 - **Static variables** (if any) are also stored here.
- **Characteristics:**
 - **Persistent:** Retains values throughout the program's execution.
 - **Initialized Data Segment:** Stores initialized global variables.
 - **Uninitialized Data Segment (BSS):** Stores uninitialized global variables (typically zero-initialized).

4.19.4.5 5. Heap Section

- Stores **dynamically allocated memory**:
 - Memory allocated at runtime (e.g., via `malloc()` in C or `new` in C++).
 - Includes **data structures** (e.g., linked lists, trees) created during execution.
- **Behavior:**
 - **Grows upward** in memory (from lower to higher addresses).
 - Managed explicitly by the programmer (or garbage collector in some languages).

4.19.4.6 Visual Representation

- **Memory Layout of a Process:**



----- Lower Memory Addresses -----

- **Key Observation:**
 - Stack and heap grow **toward each other** in memory.
 - **Stack:** High to low addresses.
 - **Heap:** Low to high addresses.

4.19.5 Summary of Key Concepts

1. **Process Definition:**
 - A program in execution, loaded into main memory.
2. **Program vs. Process:**
 - Program: Passive (stored on disk).
 - Process: Active (executed by CPU).
3. **Multiple Processes:**
 - A single program can generate multiple processes (e.g., running `./a.out` in multiple terminals).
4. **Process Components:**
 - **Text Section:** Program code.
 - **Current Activity:** PC and processor registers.
 - **Stack:** Temporary data (function calls, locals).
 - **Data Section:** Global variables.
 - **Heap:** Dynamically allocated memory.
5. **Memory Growth Directions:**
 - Stack: Downward (high -> low addresses).
 - Heap: Upward (low -> high addresses).

4.20 What to do after process creation

4.20.1 Introduction

- This lecture explores actions taken after a process is created in an operating system.
- Focuses on:
 - How to make parent and child processes perform different tasks.
 - Mechanisms for parent processes to wait for child processes.
 - Use of `fork`, `wait`, and `exec` system calls.

4.20.2 Post-Process Creation Behavior

4.20.2.1 Default Behavior After `fork()`

- When a parent process creates a child using `fork()`:
 - The child's address space is an **exact replica** of the parent's.
 - Both processes resume execution from the statement **immediately after** the `fork()` call.
 - Both execute **identical code** unless explicitly differentiated.
- **Concurrency:**
 - Parent and child may run **concurrently** (or in parallel if multiple processors are available).
 - Execution order is **non-deterministic** unless synchronized.

4.20.2.2 Problem: Identical Execution

- By default, parent and child perform the **same task**, which is often **undesirable**.
- Need mechanisms to:

1. Differentiate tasks between parent and child.
2. Enforce execution order (e.g., parent waits for child).

4.20.3 Parent-Child Synchronization: The wait() System Call

4.20.3.1 Purpose

- Allows a parent process to **pause execution** until one or more child processes terminate.
- Ensures **ordered execution** (e.g., child completes before parent).

```
pid_t wait(int *status);
```

4.20.3.2 Function Signature

- **Header File:** sys/wait.h
- **Return Value:**
 - On success: **PID of the terminated child**.
 - On failure: -1.
- **Argument:**
 - **status:** Pointer to an integer where the child's **exit status** is stored.
 - * If NULL, the parent ignores the child's exit status.

4.20.3.3 Behavior

1. **Parent State Transition:**
 - When wait() is called, the parent moves from **running** to **waiting** state.
 - Remains blocked until a child process terminates.
2. **Child Termination:**
 - Child exits via:
 - Explicit call to exit().
 - Implicit return from main().
 - Exit status is captured in status (if status != NULL).
3. **Exit Status Interpretation:**
 - **Normal termination:** status = 0.
 - **Abnormal termination:** status = non-zero (e.g., segmentation fault, killed by signal).
4. **Multiple Children:**
 - If a parent has multiple children, wait() returns the PID of **any one terminated child**.
 - Subsequent wait() calls can track other children.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main() {
    pid_t p = fork(); // Create child process

    if (p == 0) {
```

```

    // Child process
    printf("Hello\n");
    exit(0); // Terminate child
} else {
    // Parent process
    wait(NULL); // Wait for child (ignore exit status)
    printf("Bye\n");
}
return 0;
}

```

4.20.3.4 Example Code

4.20.3.5 Output Analysis

- **With wait():**
 - Guaranteed output order: Hello (child) -> Bye (parent).
- **Without wait():**
 - Output order is **unpredictable** (race condition):
 - * Possible: Hello -> Bye or Bye -> Hello.

4.20.3.6 Key Observations

- wait(NULL): Parent waits but **does not check exit status**.
- wait(&status): Parent waits **and** inspects **status** to determine how the child terminated.

4.20.4 Differentiating Parent and Child Tasks

4.20.4.1 Method 1: Conditional Logic (Using fork() Return Value)

- fork() returns:
 - 0 to the child process.
 - **Child's PID** to the parent process.
- **Strategy:** Use if-else blocks to assign different tasks.

```

if (p == 0) {
    // Child code
} else {
    // Parent code
}

```

4.20.4.2 Method 2: Process Image Replacement (Using exec)

4.20.4.2.1 Overview

- The exec family of functions **replaces the current process image** with a new program.
- **Use Case:** Launch a different program in the child process (e.g., run /bin/ls).
- **Effect:**
 - The calling process (parent or child) is **overwritten** by the new program.
 - No return on success (only returns on error: -1).

4.20.4.2.2 exec Variants

- Defined in `unistd.h`.
- Common variants:
 - `execlp`: Searches for the binary in the `PATH` environment variable.
 - `execve`: Allows explicit environment variables.
 - `execl`, `execv`, etc.

```
int execlp(const char *path, const char *arg0, ..., NULL);
```

4.20.4.2.3 Function Signature: `execlp`

- **Arguments:**
 1. `path`: Path to the binary (e.g., `"/bin/ls"`).
 2. `arg0`, `arg1`, ...: Command-line arguments for the new program.
 - `arg0` is conventionally the program name.
 3. `NULL`: Terminates the argument list.
- **Return Value:**
 - **Success**: Does not return (process image replaced).
 - **Failure**: Returns `-1` (e.g., binary not found).

```
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t p = fork();

    if (p == 0) {
        // Child process: Replace image with /bin/ls
        execlp("/bin/ls", "ls", "-l", NULL);
        // Only reaches here if execlp fails
        perror("execlp failed");
        exit(1);
    } else {
        // Parent process
        wait(NULL); // Wait for child to finish
        printf("Child completed\n");
    }
    return 0;
}
```

4.20.4.2.4 Example: Child Process Running `/bin/ls`

4.20.4.2.5 Key Points

- **Process Identity**: The PID remains the same, but the program changes.
- **Error Handling**: Always check for `exec` failure (return value `-1`).
- **No Return on Success**: The original process ceases to exist after `exec`.

4.20.5 Comparison: Conditional Logic vs. exec

Feature	Conditional Logic (if-else)	exec Family
Purpose	Run different code blocks in parent/child.	Replace process with a new program.
Process Image	Retains original image.	Overwritten by new binary.
Flexibility	Limited to same program logic.	Can run any executable (e.g., <code>ls</code> , <code>grep</code>).
Return Behavior	Continues execution after <code>fork()</code> .	No return on success (process replaced).
Use Case	Lightweight task differentiation.	Heavyweight: Launch external programs.

4.20.6 Summary of Key Concepts

1. Post-fork() Behavior:

- Parent and child initially execute the same code unless differentiated.
- Use `if-else` or `exec` to assign distinct tasks.

2. Synchronization with wait():

- Parent pauses until a child terminates.
- `wait(&status)` captures exit status (normal/abnormal).
- `wait(NULL)` ignores exit status.

3. Process Replacement with exec:

- Replaces the calling process's memory with a new program.
- Variants like `execvp` allow flexible argument passing.
- No return on success; errors return `-1`.

4. Practical Implications:

- `wait()` ensures ordered execution (e.g., child completes before parent).
- `exec` enables running external programs (e.g., shell commands) from within a process.

4.20.7 Example Workflow

1. Parent creates child via fork().

2. Child:

- Uses `exec` to run a new program (e.g., `sort`).
- Exits with status `0` (success) or non-zero (failure).

3. Parent:

- Calls `wait(&status)` to pause until child finishes.
- Checks `status` to handle child's exit (e.g., log errors).
- Continues execution (e.g., prints "Done").

4.20.8 Common Pitfalls

1. Zombie Processes:

- If parent does not `wait()` for a child, the child becomes a **zombie** (terminated but not reaped).
- Solution: Always `wait()` or use `sigchild` handlers.

2. exec Failures:

- Forgetting to check the return value of `exec` (assume success).
- Incorrect path or arguments (e.g., missing `NULL` terminator).

3. Race Conditions:

- Without `wait()`, parent and child may execute in unpredictable order.
- Example: Child writes to a file while parent reads it concurrently.

4.20.9 Header Files Reference

Function	Header File
fork()	unistd.h
wait()	sys/wait.h
exec family	unistd.h
pid_t	sys/types.h
exit()	stdlib.h